The Journal of Systems and Software 231 (2026) 112607

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Generative AI for software architecture. Applications, challenges, and future directions☆

Matteo Esposito [a], Xiaozhou Li [a], Sergio Moreschini [a,b], Noman Ahmad [a], Tomas Cerny [c], Karthik Vaidhyanathan [d], Valentina Lenarduzzi [a], Davide Taibi [a,*]

[a] University of Oulu, Finland
[b] Tampere University, Finland
[c] University of Arizona, USA
[d] Software Engineering Research Center, IIIT Hyderabad, India

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.15032395

Keywords:
Generative AI
Software architecture
Multivocal literature review
Large language model
Prompt engineering
Model human interaction
XAI

ABSTRACT

Context: Generative Artificial Intelligence (GenAI) is transforming much of software development, yet its application in software architecture is still in its infancy.
Aim: Systematically synthesize the use, rationale, contexts, usability, and challenges of GenAI in software architecture.
Method: Multivocal literature review (MLR), analyzing peer-reviewed and gray literature, identifying current practices, models, adoption contexts, reported challenges, and extracting themes via open coding.
Results: This review identifies a significant adoption of GenAI for architectural decision support and architectural reconstruction. OpenAI GPT models are predominantly applied, and there is consistent use of techniques such as few-shot prompting and retrieval-augmented generation (RAG). GenAI has been applied mostly to the initial stages of the Software Architecture Life Cycle (SALC), such as Requirements-to-Architecture and Architecture-to-Code. Monolithic and microservice architectures were the main targets. However, rigorous testing of GenAI outputs was typically missing from the studies. Among the most frequent challenges are model precision, hallucinations, ethical aspects, privacy issues, lack of architecture-specific datasets, and the absence of sound evaluation frameworks.
Conclusions: GenAI shows significant potential in software design, but there are several challenges on its way towards greater adoption. Research efforts should target designing general evaluation methodologies, handling ethics and precision, increasing transparency and explainability, and promoting architecture-specific datasets and benchmarks to overcome the gap between theoretical possibility and practical use.

Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

1. Introduction

Generative AI (GenAI) is driven by the need to create, innovate, and automate complex tasks that traditionally require human creativity. It empowers companies and individuals to unlock new possibilities, promote innovation, and improve productivity (Esposito et al., 2024a).

In software engineering, GenAI is revolutionizing the way developers design, write, and maintain code (Russo, 2024). Given its potential and benefits, the integration of GenAI within the domain of software engineering has gained increasing attention as it has a transformative potential to enhance and automate various aspects of the software development lifecycle (Sauvola et al., 2024).

Although GenAI has shown its capabilities in areas such as code generation, software documentation, and software testing (Jahić and Sami, 2024; Esposito et al., 2024b), its application in software architecture remains an emerging area of research, with ongoing debates about its effectiveness (Dhar et al., 2024a), reliability (Raghavan, 2024), and best practices (Soliman and Keim, 2025). Researching the application of GenAI in software architecture is crucial because it has the potential

☆ Editor: Heiko Koziolek.

* Corresponding author.
E-mail addresses: matteo.esposito@oulu.fi (M. Esposito), xiaozhou.li@oulu.fi (X. Li), sergio.moreschini@oulu.fi (S. Moreschini), noman.ahmad@oulu.fi (N. Ahmad), tcerny@arizona.edu (T. Cerny), karthik.vaidhyanathan@iiit.ac (K. Vaidhyanathan), valentina.lenarduzzi@oulu.fi (V. Lenarduzzi), davide.taibi@oulu.fi (D. Taibi).

https://doi.org/10.1016/j.jss.2025.112607
Received 17 March 2025; Received in revised form 12 August 2025; Accepted 26 August 2025
Available online 17 September 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

to transform the way complex systems are designed, optimized, and maintained.

However, practitioners and researchers continue to be challenged in understanding the implications, limitations, and potential benefits of GenAI for architectural tasks. To catalyze research in this area, they need a roadmap on various research directions, applications, trends, challenges, and future directions.

To better understand the existing research in this area, we **investigated the current state of research and practice on the use of GenAI in software architecture**.

Specifically, we conducted a Multivocal Literature Review (MLR) to synthesize the findings from academic literature and grey literature sources, including industry reports, blog posts, and technical documentation (Garousi et al., 2019). In particular, our goal is to understand how GenAI is used in software architecture and what the underlying rationales, models, and usage approaches are, as well as the context and practical use cases where GenAI has been adopted for software architecture. Moreover, we aim to understand research gaps highlighted by the literature and to provide an overview of possible research directions to practitioners and researchers.

Despite the growing adoption of GenAI in software engineering, several factors justify the need for a systematic investigation into its role in software architecture:

- *Emerging and Underexplored Research Area*: Although GenAI has been widely adopted in software architecture tasks, its role in software architecture remains underdeveloped (Jahić and Sami, 2024). Studies suggest that while GenAI models can help in architectural modeling and decision-making, their contributions are still in the early stages of research and adoption (Dhar et al., 2024a).
- *Lack of Systematic Evidence on Effectiveness and Reliability*: Existing work reports inconsistent findings regarding the reliability of GenAI for architectural decisions (Raghavan, 2024). Some studies indicate its potential in architectural modeling and automation, while others highlight challenges such as hallucinations, interpretability, and alignment with established architectural principles (Soliman and Keim, 2025).
- *Need for a Comprehensive Synthesis of Both Academic and Grey Literature*: Given the rapid evolution of GenAI models, grey literature, such as industry reports and practitioner blogs, provides valuable but fragmented knowledge that needs systematic integration (Kaplan et al., 2024).
- *Unclear Best Practices and Guidelines for Adoption*: Although strategies such as prompt engineering, Retrieval-Augmented Generation (RAG), and fine-tuning have been explored, there is no consensus on best practices for effectively using GenAI in different software architecture tasks (Díaz-Pace et al., 2024; Diaz-Pace et al., 2025). A structured review can help identify and formalize these practices for both researchers and practitioners (Supekar et al., 2024).
- *Increasing Industry Interest in Architectural Automation*: Enterprises are increasingly exploring AI-assisted architectural decision-making tools, yet there is still limited understanding of their practical benefits and risks (Fujitsu, 2025). The demand for explainable AI in architecture, and in particular in safety-critical domains, highlights the need for a systematic evaluation of the literature (Martelli et al., 2023).
- *Identifying Open Challenges*: Multiple research questions remain open on multiple aspects. Examples are security vulnerabilities introduced by AI-driven modifications (Raghavan, 2024), biases in architectural decision making (Johansson et al., 2024), or ethical implications of AI-generated architectural decisions (Eisenreich et al., 2024). This work will help illuminate open challenges highlighted by practitioners and researchers.

Our results reveal that, while GenAI excels at automating tasks grounded in natural language and structured templates, its integration into complex, high-stakes architectural decision-making remains limited. The prevailing utilization of these tools is predominantly oriented towards documentation and code generation, with a paucity of examples addressing system-level reasoning, trade-off analysis, or performance modeling. Furthermore, most studies evaluated GenAI tools based on usability or accuracy metrics; few addressed the impact of these tools on architectural quality attributes such as modifiability, scalability, or maintainability. These findings demonstrate an absence of rigorous evaluation.

The main contributions of this study are as follows.

- **A comprehensive synthesis** of the existing literature and industry reports to provide an overview of how GenAI is used in software architecture.
- **A classification of the GenAI models** adopted for Software Architecture based on data extracted following the open coding approach (Corbin and Strauss, 2008).
- **Identification of Common Applications, benefits, and challenges** of the application of GenAI in software architecture.
- **Identification of research gaps** and open research questions that provide recommendations for future studies and practical adoption.
- **Industry Relevance** By incorporating the grey literature, we bridge the gap between research and practice, ensuring that our findings are aligned with real-world applications.

Paper Structure: Section 2 presents the related work. Section 3 describes the study design. Section 4 presents the results obtained, and Section 5 discusses them. Section 6 highlights the threats to the validity of our study. Finally, Section 7 draws the conclusion.

2. Related work

Different works have been done to understand the extent to which large language models have been applied in software engineering (Table 1). Fan et al. (2023) performed a survey to identify how LLMs have been leveraged by different steps in the software engineering lifecycle. The work highlights that while much emphasis has been given to implementation, particularly code generation, not much work has been done in the area of using LLMs for requirements and design. This is further emphasized by Hou et al. (2024), where the authors performed a systematic literature review to understand the usage of LLMs in software engineering with a particular focus on how LLMs have been leveraged to optimize processes and outcomes. The authors analyzed 395 research articles and concluded that similar to the previous study, most of the applications of LLMs have been on software development. It is also important to note that the work only selected four relevant academic literature that leverage LLMs for software design. Thereby emphasizing the need for a multi-vocal literature review. Ozkaya (2023) provided a pragmatic view into using LLMs for Software Engineering tasks by enlisting the opportunities, associated risks, and potential challenges. The work points out challenges such as bias, data quality, privacy, explainability, etc, while describing some of the opportunities with respect to specification generation, code generation, documentation.

There have also been various secondary studies focusing on the use of LLMs for specific aspects of Software Engineering. For instance, Jiang et al. (2024) performed a systematic literature review to understand the use of LLMs for code generation. The authors selected and analyzed around 235 articles and developed a taxonomy of LLMs for code generation. Further, the work points out critical challenges and identifies opportunities to bridge the gap between research and practice of using LLMS for code generation. Wang et al. (2024), on the other hand, performed a systematic literature review to identify the different types of work that have used LLMs for software testing. It identified and

Table 1
Classification and comparison of related systematic studies.

Reference	Systematic study type	Main focus area	Identified challenges	Key findings
Hou et al. (2024)	SLR	Process optimization using LLMs	Limited software design applications	Majority use in software development phases, underscoring the need for multi-vocal studies.
Ozkaya (2023)	Hol	Risks and opportunities of LLMs in SE	Bias, data quality, privacy, explainability	Highlights potential in specification, code, and documentation generation tasks.
Jiang et al. (2024)	SLR	LLMs for code generation	Bridging research-practice gap	Taxonomy developed; outlined research-practice gaps and future opportunities.
Wang et al. (2024)	SLR	LLM applications in software testing	Integration challenges	Extensive LLM usage in testing highlighted; discussed practical integration barriers.
Marques et al. (2024)	Hol	ChatGPT in requirements engineering	Data accuracy and relevance	Provided a detailed overview of current use, challenges, and identified future directions.
Santos et al. (2024)	SLR	Generative AI impact on SE lifecycle	Overemphasis on development/testing phases	Confirmed dominance of development/testing; suggested expansion to other SE phases.
Saucedo and Rodríguez (2024)	SMS	AI for migration to microservices	Accuracy of unsupervised learning methods	Highlighted clustering as a prevalent AI technique for migrating monolithic to microservices architecture.
Fan et al. (2023)	Hol	LLMs in SE lifecycle	Limited exploration in requirements/design	Emphasis predominantly on code generation; limited attention to early SE phases.
Bucaioni et al. (2025)	SLR	AI integration with Software Architecture	Text notes real-time adaptation, trade-off analysis, etc	Vision for AI in architecture including design automation and diagnostics
Schmid et al. (2025)	SLR	AI usage in Software Architecture	Gaps in code generation from architecture, cloud-native, etc.	Key application areas; gaps in advanced prompting and evaluation
Our Work	MLR	Generative AI specifically for software architecture	Scarcity of comprehensive reviews; dominance of grey literature	Provides comprehensive insights, bridging academic and industry perspectives in generative AI applied to software architecture.

SLR - Systematic Literature Review; **SMS** - Systematic Mapping Study; **MLR** - Multivocal Literature Review; **Hol** Holistic Review.

analyzed 102 relevant studies that have used LLMs for software testing from both the software testing and LLMs perspectives. Marques et al. (2024) performed a comprehensive study to understand the application of LLMs (in particular ChatGPT) in requirements engineering. The work highlights the state of use of ChatGPT in requirements engineering and further lists the challenges and potential future work that needs to be performed in this direction. A secondary study to identify the impact of GenAI on software development activities was performed by Santos et al. (2024). Like other secondary studies on using LLMs for software engineering, this study also highlighted that most of the work has been centered around development and testing.

While to the best of our knowledge, there is a lack of secondary study on the use of GenAI applied to software architecting practices, there have been some work that leverages LLMs for various software architecting practices.

Alsayed et al. (2024) developed MicroRec, an approach that leverages state-of-the-art deep learning techniques and LLMs to recommend microservices to developers. The approach allows developers to search for microservices in service registries using natural language queries. An approach that leverages GenAI, in particular LLMs, to suggest architectural patterns from requirements was proposed by Gustrowsky et al. (2024). The proposed solution fine-tunes the Llama 2 LLM on a custom dataset of requirements and architectural patterns. The evaluation demonstrated an accuracy of 70% on the test set. Kaplan et al. (2024), on the other hand, proposed an approach that combines knowledge graphs and LLMs to support effective discovery and access to software architecture research knowledge.

Apart from the works that leverage GenAI, particularly LLMs, there have also been works that applied various AI techniques to software architecting processes/practices. Saucedo and Rodríguez (2024) performed a systematic mapping study to understand the use of AI for migrating monolithic systems to microservice-based systems. The study identified unsupervised learning, particularly clustering, as one of the most popular AI techniques used for migration based on observations from 22 primary studies.

Bucaioni et al. (2025) performed a systematic literature review and forward-looking vision for integrating AI with Software Architecture. Results shows how AI is already being used in architectural tasks like automated design, trade-off analysis, and continuous documentation updates. They identified some aspects that need to be investigated such as being able to adapt in real-time, being able to follow how changes have been made, to understand why a decision has been made, and to optimize for more than one thing at a time. They also suggests six areas to research: real-time monitoring and self-adaptation, automated documentation, context-aware reasoning, multi-objective optimization, integrated multi-level diagnostics, and robust benchmarking for practical evaluation.

Schmid et al. (2025) conducted a systematic literature review examining how LLMs are used within software architecture. They considered 18 papers identifying four primary application areas: reference architectures, classification and detection, extraction and generation, and assistant systems. The achieved results show that most approaches leverage decoder-only models like GPT variants, typically using simple prompting techniques. While LLM-based methods generally perform better than other methods, there are still several areas that are not well researched. These include generating source code from architectural designs, cloud-native architecture, and checking that things meet standards. In the future, we will be looking into ways to improve input strategies, explore more advanced prompting techniques, and make sure that we keep evaluating the technology as it develops.

Despite the active exploration of LLMs for a variety of software engineering (SE) tasks, particularly code generation, testing, requirements engineering, etc, there is a dearth of a comprehensive literature review dedicated to LLM for software architecture. Further, many of the works related to using GenAI for software design or software architecture are more available in the grey literature. Hence, in this work, we performed a multi-vocal literature review to identify the existing landscape of using GenAI for software architectural practices and processes.

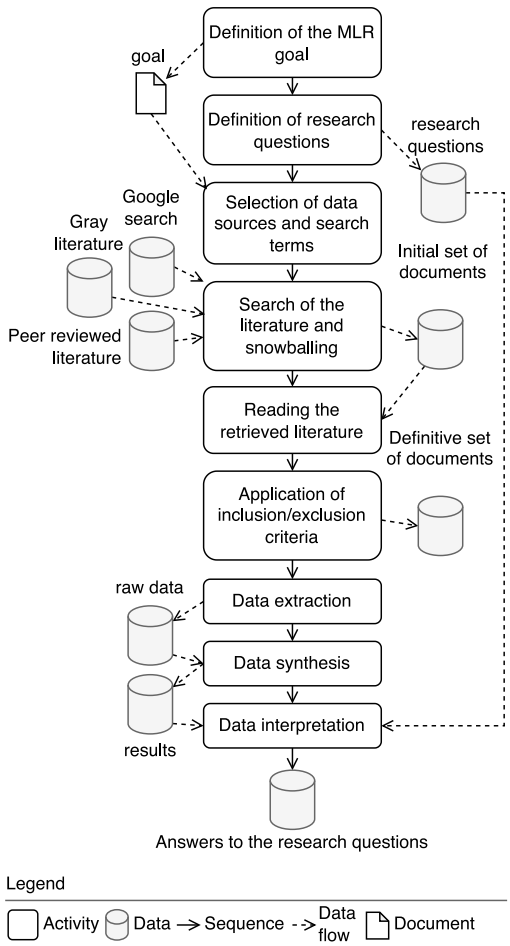

Fig. 1. The study process.

3. Methodology

This section addresses the methodology, defining the goal and research questions. It also provides the search and selection process, as well as inclusion and exclusion criteria for both peer-reviewed and grey literature. Our search strategy is presented in Fig. 1.

3.1. Goal and research questions

The goal of this MLR is to provide a comprehensive overview of GenAI's role in software architecture, from its current state to its prospects. We aim to contribute significantly to the body of knowledge in software engineering, providing actionable insights to researchers and practitioners. To carry out this research, we conducted a multivocal review of the literature (Garousi et al., 2019). Based on the objectives of our study, we defined the following research questions (RQs).

RQ$_1$

How is Generative AI utilized in software architecture, and what are the underlying rationales, models, and usage approaches?

- **RQ$_{1.1}$**. (*Why*) For what purposes are Generative AI models used in software architecture?
- **RQ$_{1.2}$**. (*What*) Which Generative AI models have been used?
- **RQ$_{1.3}$**. (*How*) How has Generative AI been applied?

In this RQ, we aim to investigate the integration of GenAI technologies in the domain of software architecture to highlight the motivations behind the adoption of these technologies, the specific models that have been employed, and the practical applications in software architecture. We try to understand the underlying rationale behind the adoption of AI models and how they contribute in practice to architectural design, maintenance, and process optimization (**RQ$_{1.1}$**). Therefore, researchers and practitioners can better assess the impact and potential of GenAI in their specific contexts.

However, in-depth investigation of the adopted GenAI models can provide a catalog of the technologies that have been implemented, providing a detailed landscape of the tools available to software architects (**RQ$_{1.2}$**).

Other important aspects to be considered are the strategies for implementing GenAI technologies in architectural practices, focusing on the types of projects that benefit from them, and the outcomes of these integrations (**RQ$_{1.3}$**).

RQ$_2$

In what contexts is Generative AI used for software architecture?

- **RQ$_{2.1}$**. (*Where*) In which software architecture life cycle phase is Generative AI applied?
- **RQ$_{2.2}$**. (*For what*) Which architectural styles or patterns are targeted?
- **RQ$_{2.3}$**. (*For what*) Which architectural maintenance tasks and quality-related activities are targeted?
- **RQ$_{2.4}$**. Which architectural analysis or modeling methods have been used to validate Generative AI outputs?
- **RQ$_{2.5}$**. To which use cases has Generative AI been applied in Software Architecture?

Once GenAI technologies have been investigated in the domain of software architecture, the next step is to explore the environments and scenarios where GenAI is integrated, mapping the conditions or settings in which these technologies are applied. Therefore, researchers and practitioners could better identify opportunities where GenAI can be used effectively, improving the architectural design process and addressing complex challenges. In particular, we identified the stages of the software architecture life cycle where GenAI tools are the most beneficial, such as requirements, design, implementation, testing, or maintenance, providing insight for the continuous integration of AI throughout the development life cycle (**RQ$_{2.1}$**). Another important aspect is to specify for which architectural styles or design patterns (e.g., microservices, monolithic architectures) (**RQ$_{2.2}$**) a GenAI model is more effective and advantageous in improving design coherence and system scalability (**RQ$_{2.3}$**). Moreover, since the benefit of adopting a new model should always be validated, it is necessary to evaluate and validate the results produced by GenAI, and architectural analysis or modeling methods have been used (**RQ$_{2.4}$**). Exploring the environments and scenarios where GenAI is integrated in architectural task led to identifying use cases where it has been implemented to highlight versatility and adaptability in different cases to solve specific problems, contribute to innovation, and drive industry advancements (**RQ$_{2.5}$**).

RQ$_3$

What challenges are identified for the use of Generative AI in software architecture?

As a last RQ, we investigate the challenges of GenAI in software architecture for which researchers and practitioners should work in the next years (**RQ**$_3$).

3.2. Search strategy

In this Section, we report the process we adopted for collecting the peer-reviewed papers and the grey literature contributions to be included in our revision.

3.2.1. Search terms

The search string contained the following search terms:

Search String

("generative AI" OR "gen AI" OR gen-AI OR genAI OR "large language model" OR "small language model*" OR LLM OR LM OR GPT* OR Chatgpt* OR Claude* OR Gemini* OR Llama* OR Bard* OR Copilot OR Deepseek)*
AND
*("software *architect*" OR "software design*" OR "software decompos*" OR"software structur*")*

In our search string, we used different terms for GenAI, such as gen AI, gen-AI, or genAI, to increase research efficiency. We used an asterisk character (*), such as software architect*, to get all possible term variations, such as plurals and verb conjugations. To increase the likelihood of finding papers that addressed our goal, we applied the search string to the title and abstract.

While surveying the state of the art, we observed that many authors referenced specific models by name in the abstract or title without using the broader term "large language model" or in short "LLM". Consequently, relying solely on generic terminology in our search string would have resulted in the omission of relevant studies. To ensure coverage of key contributions, we included a curated subset of the most widely adopted and cited models at the time of writing. These were selected based on their ranking in the Hugging Face Open LLM Leaderboard,[1] at the time this paper was originally written.

3.2.2. Bibliographic sources

For retrieving the peer-reviewed paper, we selected the list of relevant bibliographic sources following Kitchenham and Charters' recommendations (Kitchenham and Charters, 2007) since these sources are recognized as the most representative in the software engineering domain and are used in many reviews. For the white literature, we used four digital libraries: *ACM Digital Library, IEEEXplore Digital Library, Scopus, Web of Science*. Concerning the grey literature, we used five search engines: Google, Google Scholar, Bing, Yahoo, and StackOverflow (Garousi et al., 2019).

3.2.3. Inclusion and exclusion criteria

We defined the inclusion and exclusion criteria to be applied to the title and abstract (T/A), the full text (F), or both cases (All), as reported in Table 2.

We only included a paper that specifically uses LLM or GenAI for Software architecture (T/A), defines these terms (F), reports causes or factors of this phenomenon (F), proposes approaches or tools for their measurement (F), and recommends any techniques or approaches for remediation (F).

In the exclusion criteria, we excluded a paper that was not written in English (T/A), was duplicated, or had an extension already included in the review (T/A), they were beyond the scope (All), or was not accessible by an institution (T/A).

[1] https://huggingface.co/open-llm-leaderboard.

Table 2
Inclusion and exclusion criteria.

ID	Criteria	Step
I_1	Papers should specifically use LLM or Generative AI for Software architecture[a]	All
E_1	Not in English	T/A
E_2	Duplicated/extension has been included	T/A
E_3	Out of topic	All
E_4	Non peer-reviewed papers	T/A
E_5	Not accessible by institution	T/A
E_6	Papers mentioning software architecture for running LLM or GenAI	F
E_7	Papers before 15.3.2022 when the initial release of GPT-3.5 was made publicly available[b]	F

[a] The papers should genuinely be talking about LLM and SA, not just mentioning the buzzword in abstracts/discussion.
[b] https://platform.openai.com/docs/models.

3.2.4. Search and selection process for the peer-reviewed papers (white)

We conducted the search and selection process in February 2025 and included all available publications until this period. The application of the search terms returned **619 unique white papers** as reported in Table 5.

- *Testing the applicability of the inclusion and exclusion criteria:* Before implementing the inclusion and exclusion criteria, we evaluated their applicability (Kitchenham and Brereton, 2013) in ten randomly chosen articles from the retrieved paper (assigned to all authors).
- *Applying inclusion and exclusion criteria to the title and abstract:* We used the same criteria for the remaining 609 articles. Two authors read each paper, and if there was any disagreement, a third author participated to resolve the disagreement. We included a third author for 5 papers. The inter-rater agreement via the Cohen's k showed a substantial agreement. Based on the title and abstract, we selected 46 of the original 619 papers.
- *Full reading:* We performed a full read of the 46 papers included by title and abstract, applying the inclusion and exclusion criteria defined in Table 2 and assigning each article to two authors. We involved a third author for 4 papers to reach a final decision. Based on this step, we selected 27 papers as possibly relevant contributions (Cohen's k agreement: substantial agreement).
- *Snowballing:* The snowballing process (Wohlin, 2014) involved: (1) the evaluation of all articles that cited the recovered articles and (2) the consideration of all references in the recovered articles. The snowball search was performed in February 2025. We found that 11 articles were included in the final set of publications. Since our search and selection process was conducted immediately after the notification of the International Conference on Software Architecture (ICSA) 2025, we waited for the pre-print of all accepted papers to be available to avoid not including some potentially interesting contributions.
- *Quality and Assessment Criteria:* Before proceeding with the review, we checked whether the quality of the selected articles was sufficient to support our goal and whether the quality of each article reached a certain quality level. We perform this step according to the protocol proposed by Dybå and Dingsøyr (2008). To evaluate the selected articles, we prepared a checklist (Table 3) with a set of specific questions. We rank each answer, assigning a score on a five-point Likert scale (0 = poor, 4 = excellent). A paper satisfied the quality assessment criteria if it achieved a rating higher than (or equal to) 2. Among the 38 papers included in the review of the search and selection process, only 36 fulfilled the quality assessment criteria, as reported in Table 5.

Starting from the 619 unique papers, following the process, we finally included **36** papers as reported in Table 5.

Table 3
Quality assessment criteria — Peer-reviewed papers (white).

QA_s	QA
QA_1	Is the paper based on research (or is it merely a "lessons learned" report based on expert opinion)?
QA_2	Is there a clear statement of the aims of the research?
QA_3	Is there an adequate description of the context in which the research was carried out?
QA_4	Was the research design appropriate to address the aims of the research?
QA_5	Was the recruitment strategy appropriate for the aims of the research?
QA_6	Was there a control group with which to compare treatments?
QA_7	Was the data collected in a way that addressed the research issue?
QA_8	Was the data analysis sufficiently rigorous?
QA_9	Has the relationship between researcher and participants been considered to an adequate degree?
QA_{10}	Is there a clear statement of findings?
QA_{11}	Is the study of value for research or practice?

Response scale: 4 (Excellent), 3 (Very Good), 2 (Good), 1 (Fair), 0 (Poor).

3.2.5. Search and selection process for the grey literature

The search was carried out in February 2025 and included all publications available until this period. The application of the search terms returned **434 unique contributions to the grey literature** as reported in Table 5.

- *Testing the applicability of inclusion and exclusion criteria.* We used the same method adopted in the search and selection process for the peer-reviewed papers (10 papers as a test case)
- *Applying inclusion and exclusion criteria to title and abstract.* We applied the criteria to the remaining 424 papers. Two authors read each paper, and if there were disagreements, a third author participated in the discussion to resolve them. For 9 articles, we include a third author. Of the 434 initial papers, we included 77 based on title and abstract (Cohen's k agreement: almost perfect agreement).
- *Full reading.* We fully read the 77 articles included by title and abstract, applying the criteria defined in Table 2 and assigning each to two authors. We involve a third author for one paper to reach a final decision (Cohen's k agreement: almost perfect agreement). Based on this step, we selected 7 papers as possibly relevant contributions.
- *Snowballing.* The snowball search was carried out in February 2025. We found that 3 articles were included in the final set of publications.
- *Quality and Assessment Criteria.* Different from peer-reviewed literature, grey literature does not go through a formal review process, and therefore, its quality is less controlled. To evaluate the credibility and quality of the sources selected from the grey literature and to decide whether to include a source from the grey literature or not, we applied the quality criteria proposed by Garousi et al. (2019) (Table 4), considering the authority of the producer, the methodology applied, objectivity, date, novelty, impact, and outlet control. Two authors assessed each source using the aforementioned criteria, with a binary or 3-point Likert scale, depending on the criteria themselves. In case of disagreement, we discuss the evaluation with the third author, who helped provide the final assessment. We finally calculated the average of the scores and rejected sources from the grey literature that scored less than 0.5 on a scale ranging from 0 to 1. All the ten articles passed the quality criteria.

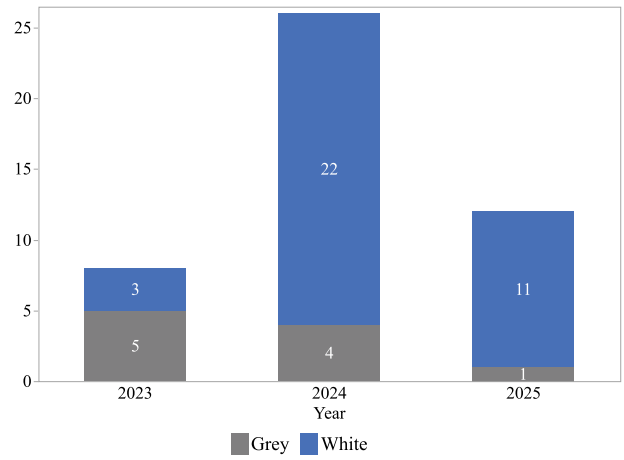

Fig. 2. Publication source trend.

Starting from the 434 unique papers, following the process, we finally included **10** grey literature papers as reported in Table 5.

3.3. Data extraction

Starting from the initial **1053 unique papers** (619 white and 434 grey), following the process, we finally included **46 papers** (36 white and 10 grey) as reported in Table 5. The data extraction form, together with the mapping of the information needed to answer each RQ, is summarized in Table 6. We extracted the data following the open coding approach (Corbin and Strauss, 2008), in which two authors extracted the information, and we involved a third author in case of disagreement. This data is exclusively based on what is reported in the papers, without any kind of personal interpretation.

4. Results

In this Section, we report the results to answer our RQs. From this section onward, we visually and textually distinguish results from white literature using **WL**, and from grey literature using **GL**.

4.1. Study context

This sub-section provides an overview of the study context in the reviewed research, including the types of studies conducted, the balance between white and grey literature, and the categories of published works (see Table 10).

Most of the works we considered belong to white literature (36; 78%) while 22% (10) to the grey (Table 7). Case studies are the most common type (28; 40%), followed by method proposals (20, 29%) and experiments (10; 14%). Tool reviews are proposed only from grey literature (7; 10%) while proof-of-concept (PoC) studies (2; 3%) interestingly are represented only by white literature. Surprisingly, we only included a few position papers (1; 2%) WL[29] and vision papers (3; 7%) (Table 8). Most of them are full papers (25; 52%), followed by short papers (7; 15%) and a few Thesis (2; 4%) (Table 9). Finally, according to Fig. 2, GenAI in SA was prominently discussed and featured in the grey literature during the start of the hype (2023), but the white literature became prominent the year after consolidating in 2025 as the main publication source for the topic.

4.2. Generative AI for software architecture: How is it used (RQ_1)

Here, we present how GenAI is currently applied in SA in terms of purpose, models used, and techniques for performance improvement, such as prompt engineering practices and the level of human interaction.

Table 4
Quality assessment criteria — Grey literature.

Criteria	Questions	Possible Answers
Authority of the producer	Is the publishing organization reputable?	1: reputable and well known organization
		0.5: existing organization but not well known, 0: unknown or low-reputation organization
	Is an individual author associated with a reputable organization?	1: true
		0: false
	Has the author published other work in the field?	1: Published more than three other work
		0.5: published 1-2 other works, 0: no other works published.
	Does the author have expertise in the area? (e.g., job title principal software engineer)	1: author job title is principal software engineer, cloud engineer, front-end developer or similar
		0: author job not related to any of the previously mentioned groups.)
Methodology	Does the source have a clearly stated aim?	1: yes
		0: no
	Is the source supported by authoritative, documented references?	1: references pointing to reputable sources
		0.5: references to non-highly reputable sources
		0: no references
	Does the work cover a specific question?	1: yes 0.5: not explicitly 0: no
Objectivity	Does the work seem to be balanced in presentation	1: yes 0.5: partially 0: no
	Is the statement in the sources as objective as possible? Or, is the statement a subjective opinion?	1: objective 0.5 partially objective 0: subjective
	Are the conclusions free of bias or is there vested interest? E.g., a tool comparison by authors that are working for a particular tool vendor	1 = no interest
		0.5: partial or small interest 0: strong interest
	Are the conclusions supported by the data?	1: yes 0.5: partially 0: no
Date	Does the item have a clearly stated date?	1: yes 0: no
Position w.r.t. related sources	Have key related GL or formal sources been linked to/discussed?	1: yes 0: no
Novelty	Does it enrich or add something unique to the research?	1: yes 0.5: partially 0: no
Outlet type	Outlet Control	1: high outlet control/ high credibility: books, magazines, theses, government reports, white papers
		Moderate outlet control/ moderate credibility: annual reports, news articles, videos, Q/A sites (such as StackOverflow), wiki articles
		0: low outlet control/low credibility: blog posts, presentations, emails, tweets

4.2.1. Why GenAI in SA ($RQ_{1.1}$)

Architectural decision support is the purpose most frequently investigated in the reviewed studies, appearing in 38% (18) of them (Table 11 - $\mathbf{RQ}_{1.1}$). This suggests that the primary focus of current research on GenAI in software architecture is its application in assisting architectural decision-making. For example, WL[3] uses GenAI to generate microservice names, while GL[8] uses it to support software design and requirement engineering, and WL[18] uses it to guide software architects in making architectural decisions. Similarly, the second most frequent purpose for using GenAI in the case of reverse engineering for architectural reconstruction appears in 19% (9) of the cases. On the other hand, the least explored uses are Reverse Engineering for Traceability (GL[11]) and Migration & Re-engineering (WL[30]), each of which appeared only in 2% (1) of the studies (Table 11 - $\mathbf{RQ}_{1.1}$).

Table 5
Search and selection process.

Step	#
Retrieval from white sources (unique papers)	**619**
-Reading by title and abstract	−573
-Full reading	−19
-Snowballing	+11
-Quality assessment	−2
Primary studies	**36**
Retrieval from grey sources (unique papers)	**434**
-Reading by title and abstract	−357
-Full reading	−70
-Snowballing	+3
Primary studies	**10**

Table 6
Data extraction.

Data	RQ	Outcome
Work category	na	List of category
Methods		List of methodological approaches
Author		First and last name Affiliation
Publication sources		Peer-reviewed literature (white) Grey literature Publication name Publication type (e.g., journal) Publication year
GenAI usage	$RQ_{1.1}$	Purpose (why)
	$RQ_{1.2}$	Model (what)
	$RQ_{1.3}$	How
GenAI usage context	$RQ_{2.1}$	SALC phase
	$RQ_{2.2}$	For what architectural styles or patterns
	$RQ_{2.3}$	For what architectural maintenance/quality-related tasks
	$RQ_{2.4}$	Architecture analysis/modeling method
Use case	$RQ_{2.5}$	List of use cases Analyzed systems Programming languages
Challenges	RQ_3	List of challenges

Table 7
White and grey literature distribution.

Code	PaperID	#	%
White	WL[1], WL[2], WL[3], WL[4], WL[5], WL[6], WL[7], WL[8], WL[9], WL[10], WL[11], WL[12], WL[35], WL[13], WL[14], WL[15], WL[18], WL[17], WL[9], WL[19], WL[20], WL[21], WL[22], WL[23], WL[16], WL[24], WL[25], WL[26], WL[27], WL[28], WL[30], WL[31], WL[32], WL[33], WL[34], WL[36]	36	78%
Grey	GL[1], GL[2], GL[3], GL[4], GL[29], GL[5], GL[6], GL[7], GL[8], GL[10]	10	22%

1. $RQ_{1.1}$ (Why GenAI in SA)

LLMs are primarily used for architectural decision support (38%) and reverse engineering (21%), with less focus on tasks like migration, re-engineering, and traceability.

Table 8
Study type.

Code	PaperID	#	%
Case Study	WL[2], WL[3], WL[5], WL[6], WL[7], WL[8], WL[10], WL[14], WL[15], WL[18], WL[17], WL[20], WL[22], WL[23], WL[16], WL[24], WL[25], WL[26], WL[27], WL[29], WL[28], WL[30], WL[31], WL[33], WL[34], WL[36],GL[8], GL[9]	28	40%
Experiment	WL[1], WL[4], WL[9], WL[11], WL[12], WL[19], WL[21], WL[16], WL[31], GL[1]	10	14%
Exploratory Study	WL[13], WL[4]	2	3%
Method Proposal	WL[1], WL[6], WL[7], WL[8], WL[9], WL[10], WL[11], WL[12], WL[35], WL[18], WL[17], WL[20], WL[24], WL[26], WL[32], WL[33], WL[36], GL[1], GL[8], GL[9]	20	29%
PoC	WL[35], WL[26]	2	3%
Survey	WL[14]	1	1%
Tool Review	GL[2], GL[3], GL[4], GL[5], GL[6], GL[7], GL[10]	7	10%

Table 9
Study category.

Code	PaperID	#	%
Blog Post	GL[2], GL[6], GL[7], GL[10]	4	9%
Full Paper	WL[1], WL[3], WL[4], WL[5], WL[7], WL[8], WL[9], WL[11], WL[12], WL[35], WL[13], WL[15], WL[18], WL[17], WL[20], WL[21], WL[16], WL[24], WL[27], WL[28], WL[31], WL[32], WL[33], WL[36], GL[1]	25	54%
Industry Report	WL[30]	1	2%
Position Paper	WL[29]	1	2%
Short Paper	WL[2], WL[14], WL[19], WL[22], WL[23], WL[25], WL[34]	7	15%
Thesis	GL[9], GL[8]	2	4%
Vision Paper	WL[6], WL[10], WL[26]	3	7%
White Paper	GL[4], GL[5]	2	4%
Youtube Video	GL[3]	1	2%

4.2.2. GenAI model used ($RQ_{1.2}$)

OpenAI GPT models are the ones that rule the roost and were utilized in 62% (105) of the articles, followed by Google's models (15; 9%) (Table 12 - $\mathbf{RQ}_{1.2}$). Surprisingly, the recently published open-source model DeepSeek has already been applied in two works. It is also worth noting that on-demand cloud-based models are by far the favorable option in place of on-premises due to their resource requirements.

Considering the evolution over time of the AI model providers, it is evident that OpenAI has a consistent prominence. Nevertheless, newer models such as DeepSeek and Qwen gained traction in 2024 and 2025, highlighting a shift in attention towards emerging alternatives. This trend is also confirmed in the increasing presence of models from diversified providers, i.e., miscellaneous category, including specific open source alternatives such as LLaMa (Fig. 3 - $\mathbf{RQ}_{1.2}$).

Table 10
Publication sources.

Sources name	Type	Count	Years
AIM Research	Research Institution	1	–
Communications in Computer and Information Science	Book Series	1	–
Design Society	Society Publication	1	–
Electronics (Switzerland)	Journal	1	–
European Conference on Pattern Languages of Programs, People and Practices	Proceedings	1	–
European Conference on Software Architecture	Conference	1	2024
Human-Computer Interaction	Journal	1	–
IEEE International Conference on Software Quality Reliability and Security Companion (QRS-C)	Proceedings	1	2023
IEEE International Conference on Data and Software Engineering (ICoDSE)	Proceedings	1	2023
IEEE International Requirements Engineering Conference (RE)	Conference	1	2024
IEEE International Conference on Software Architecture (ICSA)	Conference	12	2024, 2025
IEEE International Conference on Software Architecture Companion (ICSA-C)	Conference	3	2024
IEEE Software	Journal	1	–
IEEE/ACM Workshop on Multi-disciplinary Open and RElevant Requirements Engineering (MO2RE)	Workshop	1	2024
Information Technology	Journal	1	–
Institutional Website	Website	7	–
International Conference on Software Engineering	Proceedings	1	–
International Workshop on Designing Software	Workshop	1	2024
Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)	Book Series	1	–
Medium	Online Media	2	–
Methods	Journal	1	–
SN Computer Science	Journal	1	–
Studies in Computational Intelligence	Book Series	1	–
YouTube	Online Media	1	–

Table 11
Purpose of the LLM - ($RQ_{1.1}$).

Code	PaperID	Count[a]	%
Architectural Decision Support	WL[2], WL[3], WL[5], WL[7], WL[8], WL[12], WL[14], WL[18], WL[21], WL[22], WL[32], GL[2], GL[3], GL[9], GL[5], GL[6], GL[8], GL[7]	18	38%
Reverse Engineering/Architectural Reconstruction	WL[9], WL[15], WL[19], WL[20], WL[24], WL[26], WL[28], GL[4], GL[10]	9	19%
Architecture Generation	WL[1], WL[6], WL[10], WL[35], WL[17], WL[27], WL[29], WL[33], WL[36]	9	19%
Quality Assessment	WL[9], WL[19], WL[20], WL[24]	4	9%
Software Comprehension	WL[16], WL[25], WL[31]	3	7%
Requirement Engineering	WL[23], WL[34]	2	4%
Migration & Re-engineering	WL[30]	1	2%
Reverse Engineering/Traceability	WL[11]	1	2%

[a] One paper can have more than one purpose.

2. $RQ_{1.2}$ (GenAI Model Used)

OpenAI GPT models dominate (62%) the research landscape, while alternatives such as Google LLMs and LLaMA models are significantly less employed.

4.2.3. How genai is used ($RQ_{1.3}$)

Among the techniques to enhance the capabilities and performance of GenAI, Fine-Tuning is applied in 12% (6) of the studies, that is, some researchers have chosen to fine-tune LLMs for specific architectural tasks with additional training. In particular, WL[4] used Fine-Tuning to align the LLM in generating serverless functions. RAG, including proprietary variants, is applied in 20% (10) of the studies, suggesting that applying external knowledge sources is a common method to improve LLM performance in software architecture contexts. For example, WL[6] used RAG and Fine-Tuning to retrieve architecture knowledge management information and align such models to their needed task (Table 13 - $\mathbf{RQ}_{1.3}$).

A large percentage of studies (13; 25% for Prompt Engineering and 24; 48% for Model Enhancements) did not report any data. Conversely, 26% (12) reported that no improvements were applied, and the models were run as they were. Therefore, our findings reveal that while fine-tuning and RAG methods are explored, most studies do not document their method of improvement or apply the off-the-shelf models without any modifications (Table 13 - $\mathbf{RQ}_{1.3}$).

Most specifically, prompt engineering is also used to quickly align LLMs to a new task (Esposito et al., 2024c). The most widely used technology is the few-shot prompt, present in 31% (16). This shows that researchers use numerous examples to a great extent to allow LLMs to produce more precise and contextual architectural output. In contrast, one-shot prompting is the least used, with the technique mentioned in only 2% (1) of the research, suggesting that a single occurrence is infrequent in this field. Zero-shot prompting occurs in 12% (6) of the studies, at moderate frequency, where the researchers solely utilize the pre-training knowledge of the model without additional context. As an example, GL[5] employed the three techniques to evaluate LLM applications in modernizing the architecture of legacy systems. Finally, in the spectrum of reasoning enhancements, Chain-of-thought (CoT) prompting appears only in 8% (4) of the cases. WL[36] employs such a technique when evaluating an LLM-based pipeline from requirements to code.

Most studies involve some form of human interaction with the model (39; 85%), and this indicates that our community is prone to involve human observation, validation, or supplementation when

Table 12
LLM Models - ($RQ_{1.2}$).

Model Family	Model	PaperID	Count[a]	% (Model)	% (Family)
OpenAI	GPT	WL[1], WL[2], WL[3], WL[4], WL[5], WL[6], WL[7], WL[8], WL[9], WL[10], WL[11], WL[12], WL[35], WL[13], WL[14], WL[15], WL[18], WL[17], WL[19], WL[20], WL[8], WL[21], WL[23], WL[16], WL[24], WL[25], WL[26], WL[27], WL[28], WL[30], WL[31], WL[33], WL[34], WL[36], GL[2], GL[3], GL[9], GL[5], GL[6], GL[7], GL[10]	39	23%	62%
	GPT-4	WL[1], WL[5], WL[6], WL[8], WL[9], WL[11], WL[12], WL[35], WL[13], WL[14], WL[18], WL[17], WL[20], WL[16], WL[24], WL[26], WL[28], WL[30], WL[33] GL[8]	21	13%	
	ChatGPT	WL[2], WL[3], WL[4], WL[19], WL[21], WL[27], WL[34], WL[36], GL[2], GL[3], GL[9], GL[5], GL[7]	14	8%	
	GPT-3	WL[5], WL[6], WL[7], WL[14], WL[15], WL[23], WL[25], WL[28], WL[31]	9	5%	
	GPT-3.5	WL[5], WL[6], WL[7], WL[15], WL[23], WL[25], WL[28], WL[31]	8	5%	
	GPT-4o	WL[1], WL[8], WL[9], WL[11], WL[12], WL[24], WL[33]	7	4%	
	GPT-4o-mini	WL[1], WL[8], WL[9]	3	2%	
	GPT-2	WL[5], WL[6]	2	1%	
	GPT-3.4	WL[14]	1	1%	
	GPT-4 Turbo	WL[16]	1	1%	
Google's LLM	Bard	WL[17], WL[19], WL[28], GL[3], GL[5], GL[10]	6	4%	9%
	Gemini	WL[24], WL[27], GL[10]	3	2%	
	Google Bard	WL[19], WL[28], GL[5]	3	2%	
	Bert	WL[6]	1	1%	
	Gemini 1.5	WL[24]	1	1%	
	Google Gemini	WL[27]	1	1%	
LLaMA	LLaMA	WL[10], WL[11], WL[35], WL[20], WL[29], GL[1], GL[9]	7	4%	8%
	LLaMA-3	GL[9]	2	1%	
	Llama 3.1	WL[11]	1	1%	
	LLaMA-2	GL[1]	1	1%	
	Code Llama	GL[29]	1	1%	
	Codellama 13b	WL[11]	1	1%	
DeepSeek	DeepSeek-Coder	WL[4]	1	1%	1%
	DeepSeek-V2.5	WL[1]	1	1%	
CodeQwen	CodeQwen	WL[1], WL[4]	2	1%	2%
	CodeQwen1.5-7B	WL[1]	1	1%	
GitHub Copilot	Copilot	WL[27], GL[8], GL[5], GL[6]	4	2%	2%
Mistral	Mistral	GL[1], GL[23]	2	1%	2%
	Mistral 7b	WL[23]	1	1%	
T0/T5 Derivatives	T5	WL[5], WL[6], WL[29]	3	2%	6%
	Flan-T5	WL[5], WL[6]	2	1%	
	T0	WL[5], WL[6]	2	1%	
	CodeT5	WL[29]	1	1%	
	CodeWhisperer	GL[8]	1	1%	
Miscellaneous	Adobe Firefly	GL[6]	1	1%	1%
	Claude AI	WL[30]	1	1%	
	Codex	WL[29]	1	1%	
	Codium	GL[5]	1	1%	
	Cursor	GL[5]	1	1%	
	Falcon	WL[10]	1	1%	
	k8sgpt	GL[5]	1	1%	
	Mutable.AI	GL[5]	1	1%	
	N.A	WL[22], GL[4]	2	1%	
	Phi-3	GL[1]	1	1%	
	Replit	GL[5]	1	1%	
	Robusta ChatGPT bot	GL[5]	1	1%	
	Tabnine	GL[5]	1	1%	
	Unknown	WL[32]	1	1%	
	Yi	WL[10]	1	1%	

[a] One paper can have more than one model.

using LLMs for software architecture purposes. This indicates that fully autonomous AI-driven architectural decisions are not yet prevalent, but human participation is still significant in guiding, validating, or improving LLM-generated results. For example, WL[7] leverages human interaction by providing a chat-based environment to provide AI-based support to novice architects to refine design decisions.

No human interaction has been reported for 15% (7) of the studies, and the models existed without direct human intervention. The breakdown shows a high preference for interactive approaches, validating that LLMs in software development are used primarily as auxiliary tools and not as standalone decision-makers (Table 13 - **$RQ_{1.3}$**) (see Fig. 4).

> 3. $RQ_{1.3}$ (How GenAI is used)
>
> Few-shot prompting (31%) is the most common technique, RAG (22%) is frequently used for model enhancement, and 85% of the studies involve human interaction, emphasizing the assistive rather than autonomous role of LLM.

Table 13
How GenAI is used ($RQ_{1.3}$).

	Code	PaperID	Count[a]	%
Prompt Engineering	Few-Shot	WL[1], WL[4], WL[5], WL[7], WL[9], WL[12], WL[35], WL[13], WL[18], WL[17], WL[20], WL[24], WL[30], WL[33], GL[5], GL[8]	16	31%
	Unspecified	WL[10], WL[9], WL[21], WL[16], WL[27], WL[32], GL[1], GL[2], GL[3], GL[4], GL[6], GL[7], GL[10]	13	25%
	None	WL[2], WL[3], WL[6], WL[14], WL[15], WL[19], WL[22], WL[23], WL[25], WL[26], WL[28], WL[34]	12	23%
	Zero-Shot	WL[4], WL[5], WL[7], WL[30], WL[31], GL[5]	6	12%
	Chain-of-Thought	WL[8], WL[11], WL[36], GL[29]	4	8%
	One-Shot	WL[30]	1	2%
Model Enhancements	Unspecified	WL[1], WL[8], WL[9], WL[11], WL[12], WL[35], WL[13], WL[18], WL[17], WL[21], WL[24], WL[27], WL[30], WL[29], WL[28], WL[31], WL[32], WL[33], GL[2], GL[3], GL[6], GL[7], GL[8], GL[10]	24	48%
	RAG	WL[6], WL[7], WL[10], WL[20], WL[23], WL[16], GL[1], GL[4], GL[9], GL[5]	10	20%
	None	WL[2], WL[3], WL[14], WL[15], WL[19], WL[22], WL[25], WL[26], WL[34]	9	18%
	Fine-Tuning	WL[4], WL[5], WL[6], WL[36], GL[1], GL[5]	6	12%
	Proprietary RAG	GL[4]	1	2%
Human Model Interaction	Yes	WL[1], WL[2], WL[4], WL[6], WL[7], WL[8], WL[9], WL[10], WL[11], WL[12], WL[35], WL[13], WL[14], WL[15], WL[18], WL[17], WL[19], WL[20], WL[21], WL[22], WL[23], WL[16], WL[24], WL[26], WL[27], WL[30], WL[31], WL[32], WL[33], WL[34], WL[36], GL[1], GL[2], GL[3], GL[6], GL[9], GL[7], GL[8], GL[10]	39	85%
	No	WL[3], WL[5], WL[25], WL[29], WL[28], GL[4], GL[5]	7	15%
Model used as-is		WL[2], WL[3], WL[6], WL[14], WL[15], WL[19], WL[22], WL[23], WL[25], WL[26], WL[28], WL[34]	12	26%

[a] One paper can have more than one usage.

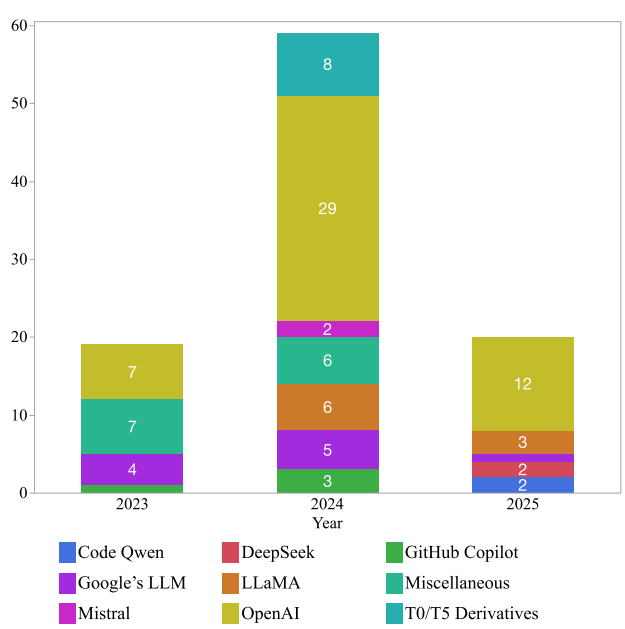

Fig. 3. LLM Vendor Trend ($RQ_{1.2}$).

4.3. Generative AI for software architecture: In which context (RQ_2)

This section presents the different contexts in which GenAI is applied within the software architecture. Specifically, we examine its role across various phases of the Software Architecture Lifecycle (SALC), the architectural styles and patterns it supports, and the validation methods used to assess its outputs.

4.3.1. SALC phases ($RQ_{2.1}$)

Regarding the use of GenAI across SALC (Table 14 and Fig. 5 - $\mathbf{RQ}_{2.1}$), the requirement-to-architecture (Req-to-Arch) is the most frequently targeted phase, as mentioned in 40% (24) of the papers. This suggests that LLMs are frequently used to fill in the requirement and architectural design gap, to assist in mapping textual specifications into formal architectural representations. In fact, WL[2] leveraged GenAI for collaborative architectural design to assist practitioners in designing the SA from requirements. Similarly, WL[3] used ChatGPT to generate microservice names (architecture) based on the requirements.

Following this, Architecture-to-Code (Arch-to-Code) is also a compelling use case, accounting for 32% (19) of the research. This indicates a significant focus on using LLMs to automate or help in mapping architectural designs to implementation-level code. Following the same logic, WL[4] used GenAI to generate a serverless function (code) from the architectural specification. However, a peculiar instance and the least explored is Architecture-to-Architecture (Arch-to-Arch) transitions, which only 3% (2) of the research covers, indicating the lack of current community interest in enhancing, migrating, or converting architectures using LLMs. In line with this, WL[20] refactored the architectural smells using LLMs such as GPT-4 and LLaMA, while WL[24] used Gemini 1.5 and GPT-4o to recommend resolutions of architectural violations.

On the other hand, code-to-architecture (8; 13%) and requirement-to-architecture-to-code (7; 12%) are fairly represented. The former is

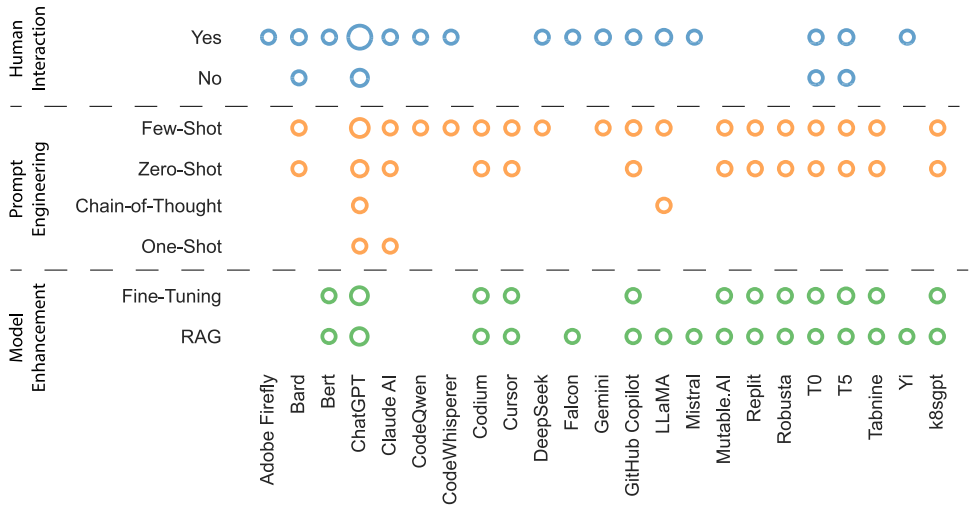

Fig. 4. How GenAI is used.

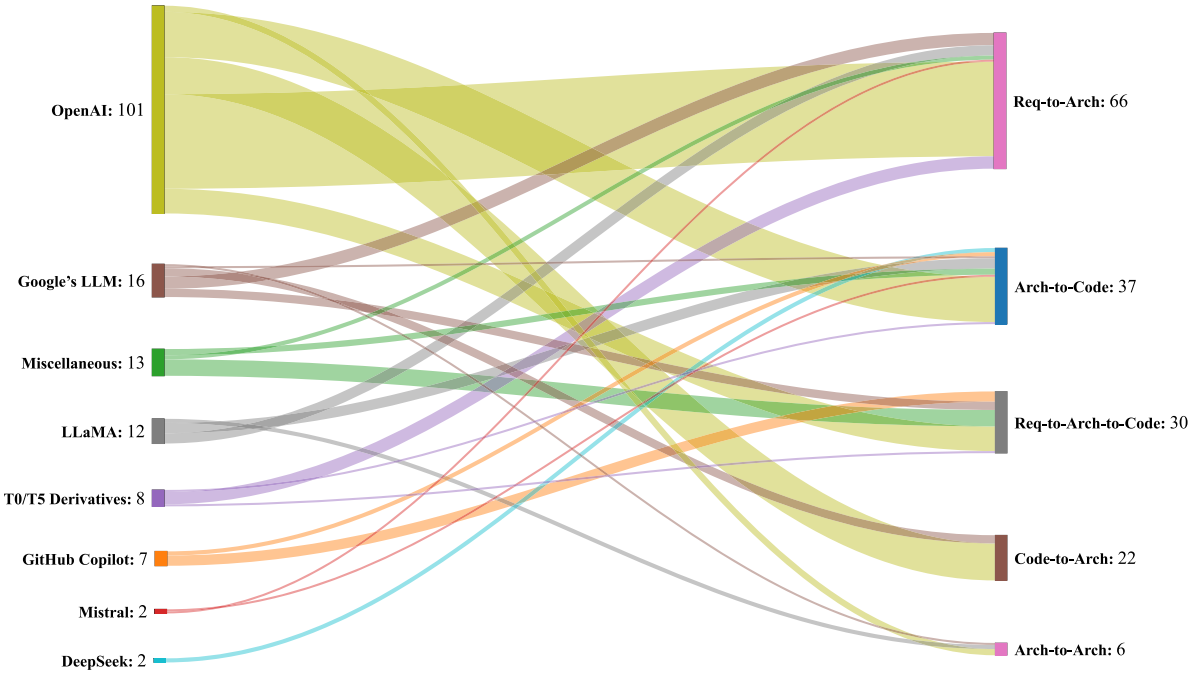

Fig. 5. Sankey plot connecting LLM Models to SALC phase.

indicative of efforts towards reverse engineering existing codebases for architectural purposes. Consistent with this approach, WL[8] experimented with developing LLM-based architecture agents that could improve architecture decision-making starting from code, while GL[4] presented its LLM-based tool to perform the architectural reconstruction.

The requirement-to-architecture-to-code illustrates efforts to optimize the entire process from requirements to architecture to code generation. Using this SALC arch, GL[3] presented in its video tutor an LLM-based copilot of such a SALC arch. Similarly, in a position paper, WL[22] presented an assisted architecture LLM based on LLM-based software on LLM-based one requirement.

The distribution of studies indicates that the significant use of LLMs is at the beginning of the SALC, e.g., during requirement analysis as well as architectural design, with less effort going towards changing or reorganizing existing architectures.

> 4. $RQ_{2.1}$ (SALC Phases)
>
> LLMs are most frequently applied in the Requirement-to-Architecture (40%) and Architecture-to-Code (32%) transitions, while Architecture-to-Architecture (3%) is the least explored.

4.3.2. *Architectural styles and patterns ($RQ_{2.2}$)*

Concerning the architectural styles and patterns to which LLMs have been applied, monolithic architectures are mentioned most frequently,

Table 14
Use of LLMs in the Software Architecture Life Cycle - ($RQ_{2.1}$).

Code	PaperID	Count[a]	%
Req-to-Arch	WL[2], WL[3], WL[5], WL[6], WL[7], WL[10], WL[35], WL[14], WL[18], WL[17], WL[22], WL[23], WL[32], WL[33], WL[34], WL[36], GL[2], GL[3], GL[9], GL[5], GL[6], GL[7], GL[8], GL[10]	24	40%
Arch-to-Code	WL[1], WL[4], WL[9], WL[11], WL[12], WL[13], WL[8], WL[21], WL[22], WL[27], WL[30], WL[29], WL[31], WL[36], GL[1], GL[3], GL[5], GL[6], GL[7]	19	32%
Code-to-Arch	WL[8], WL[15], WL[19], WL[16], WL[25], WL[26], WL[28], GL[4]	8	13%
Req-to-Arch-to-Code	WL[22], WL[36], GL[3], GL[5], GL[6], GL[7], GL[8]	7	12%
Arch-to-Arch	WL[20], WL[24]	2	3%

[a] One paper can have more than one purpose.

Table 15
Use of LLMs for Architectural style and patterns - ($RQ_{2.2}$).

Code	PaperID	Count[a]	%
Unspecified	WL[2], WL[5], WL[6], WL[7], WL[8], WL[10], WL[11], WL[35], WL[13], WL[15], WL[18], WL[17], WL[20], WL[8], WL[22], WL[23], WL[16], WL[24], WL[27], WL[29], WL[31], WL[33], WL[34], WL[36], GL[1], GL[2], GL[3], GL[9], GL[5], GL[6], GL[7], GL[10]	32	68%
Monolithic	WL[14], WL[19], WL[25], WL[26], WL[28], WL[30], GL[4]	7	15%
Microservices	WL[3], WL[9], WL[21]	3	6%
Design Patterns	WL[32]	1	2%
Layered Architecture	WL[26]	1	2%
Model-Based Architecture	WL[12]	1	2%
Self-Adaptive Architecture	WL[1]	1	2%
Serverless	WL[4]	1	2%

[a] One paper can have more than one use of LLMs.

appearing in 15% (7) of the articles (Table 15 - **RQ**$_{2.2}$). This suggests that LLMs are applied primarily in the understanding, analysis, or modernization of monolithic systems. In fact, WL[26] used LLM to perform architectural recovery from a legacy monolithic system to understand the program.

As expected, microservices also have a strong appearance and studies investigating their architectural aspects in 6% (3) of the studies.

The purpose of preserving the microservice architecture varies. For example, WL[21] uses LLM to analyze the code of a microservice-based system to answer architectural questions related to its designs (program comprehension). Similarly, WL[19] focused on the identification of antipatterns in a microservice-based system.

Other trends, such as Self-Adaptive Architecture, Serverless, Layered Architecture, and Model-Based Architecture, only appear erratically, each in 2% (1) of the studies, showing low research interest in these architectural styles.

An overwhelming 68% (32) of the research failed to include any data on architectural styles or trends, and it can be inferred that the majority of the work carried out on LLMs within software architecture does not necessarily correlate their conclusions or base the focus on a certain architectural style.

Such an asymmetrical distribution demonstrates that although the focus is given to some of the architectural schools, especially monolithic and microservices, others are left unexplored regarding the application of LLMs.

> 5. $RQ_{2.2}$ (Architectural Styles and Patterns)
>
> LLMs mainly target monolithic (15%) and microservices architectures, with 68% of studies omitting style details.

4.3.3. Quality and maintenance tasks ($RQ_{2.3}$)

Concerning quality aspects, 38% of the works explicitly discuss antipattern detection using methods such as LLM-based architectural smell refactoring, AI-based detection, and rule-based learning (**RQ**$_{2.3}$). In particular, WL[19] and WL[20] use LLM to detect antipatterns.

Concerning refactoring as a means of removing smells and improving overall software quality, WL[18] and WL[32] use LLM to aid in refactoring efforts. Moreover, WL[13] are the only authors who use an external tool (EM-Assist) to aid in refactoring, in conjunction with LLMs.

Similarly, studies that perform architectural reconstruction rely on LLM to achieve this. More specifically, WL[15] used LLM to map code components to a specific architecture, while WL[26] used LLM to recover the deductive software architecture. Finally, only WL[20] reported the use of external tools, validating the observation that LLMs are increasingly being used to recover architectural knowledge and are decreasing in strictly classical tools.

> 6. $RQ_{2.3}$ (Quality & Maintenance Tasks)
>
> 38% of studies use LLMs for antipattern detection, refactoring (WL[18], WL[32]), and architectural reconstruction (WL[15], WL[26]). Few integrate external tools, suggesting that LLMs are replacing traditional recovery methods.

4.3.4. Architecture modeling and validation methods ($RQ_{2.4}$)

Similarly to programming languages, we can represent SA via many architectural languages (AL). Among such AL, UML (Unified Modeling Language) is most commonly applied as a notation in 17% (8) of the studies (Table 16 - **RQ**$_{2.2}$) thus assessing UML as the still dominant modeling language for studies studying LLM due to its versatility in software design and architecture documentation (Esposito et al., 2024c). For example, WL[33] used LLM to generate UML component diagrams from informal specifications.

The remaining modeling approaches, i.e., C4, ADR (Architecture Decision Records), SysML, and Knowledge Graphs (KG), each appear only in one study WL[5] (2%), indicating little exploration of other architectural modeling notations. In particular, WL[5] uses ADR while using LLM to generate architectural design decisions with LLM. In contrast, WL[35] investigated automating architecture generation using LLMs in Model-Based Systems Engineering using SysML as the modeling language. GL[4] used KG for LLM-based architectural reconstruction. Finally, WL[14] used a combination of UML and C4 for LLM-based assisted architectural decision-making.

Most of the studies (35; 74%) did not report any data on the use of architectural modeling languages, suggesting that much research on LLM in software architecture does not necessarily use or elaborate formal modeling approaches. The prevalence of UML and the non-wider deployment of rival model languages suggest there is still sufficient scope for extension research combining LLMs and architected presentation forms.

Table 16
Architectural modeling language - ($RQ_{2.2}$).

Code	PaperID	Count[a]	%
Unspecified	WL[1], WL[3], WL[4], WL[6], WL[7], WL[8], WL[9], WL[10], WL[11], WL[12], WL[13], WL[15], WL[18], WL[19], WL[20], WL[21], WL[23], WL[16], WL[24], WL[25], WL[26], WL[27], WL[29], WL[28], WL[30], WL[31], WL[32], GL[1], GL[2], GL[9], GL[5], GL[6], GL[7], GL[8], GL[10]	35	74%
UML	WL[2], WL[14], WL[17], WL[22], WL[33], WL[34], WL[36], GL[3]	8	17%
ADR	WL[5]	1	2%
C4	WL[14]	1	2%
Knowledge Graph	GL[4]	1	2%
SysML	WL[35]	1	2%

[a] One paper can have more than one language.

Table 17
Model-Driven Engineering (MDE) - ($RQ_{2.2}$).

Code	PaperID	Count	%
Unspecified	WL[2], WL[3], WL[4], WL[5], WL[6], WL[7], WL[8], WL[9], WL[10], WL[11], WL[35], WL[13], WL[14], WL[15], WL[29], WL[17], WL[19], WL[20], WL[21], WL[22], WL[23], WL[16], WL[26], WL[27], WL[28], WL[30], WL[31], WL[33], WL[34], WL[36], GL[1], GL[2], GL[3], GL[4], GL[9], GL[5], GL[6], GL[7], GL[8], GL[10]	40	87%
IoT Architecture Generation	WL[1]	1	2%
Low-code Platform Consistency	WL[12]	1	2%
UML Component Diagram Generation	WL[32]	1	2%
Source Code to Architecture Mapping	WL[18]	1	2%
Architectural Conformance Recommender	WL[24]	1	2%
Deductive Software Architecture Recovery	WL[25]	1	2%

On the topic of architectural design language, five studies reported using some form of Model Driven Engineering (MDE) (Table 17 - **$RQ_{2.2}$**). More specifically, WL[1] used MDE for the generation of the IoT architecture, while WL[12] for low-code platform consistency, WL[32] generated UML component diagrams, WL[18] mapping of the source code to the architecture,WL[24] provided architectural conformance recommendations, and GL[25] deductive architecture recovery, each of which occurs in 2% (1) of the articles. However, 87% (40) of the articles did not contain information on the utilization of MDE, so while there is evidence of research that uses LLM for MDE applications, the topic is still fairly unexplored compared to other architectural activities.

93% (43) of the studies report that no information was provided on the LLM model output validation techniques (Table 18 - **$RQ_{2.4}$**) while only three of them report how they evaluated the LLM model output. In particular, WL[10] used ATAM (Architecture Tradeoff Analysis Method), while WL[2] used SAAM (Software Architecture Analysis Method) and WL[21] used static analysis. Hence, our findings suggest that formal assessment methods are still not in common practice, and most studies do not explicitly validate their AI-generated architectural designs.

Table 18
Architecture analysis method — Adopted generative AI Outputs validation methods - ($RQ_{2.3}$).

Code	PaperID	Count	%
Unspecified	WL[1], WL[3], WL[4], WL[5], WL[6], WL[7], WL[8], WL[9], WL[11], WL[12], WL[35], WL[13], WL[14], WL[15], WL[18], WL[17], WL[19], WL[20], WL[22], WL[23], WL[16], WL[24], WL[25], WL[26], WL[27], WL[29], WL[28], WL[30], WL[31], WL[32], WL[33], WL[34], WL[36], GL[1], GL[2], GL[3], GL[4], GL[9], GL[5], GL[6], GL[7], GL[8], GL[10]	43	93%
ATAM	WL[10]	1	2%
SAAM	WL[2]	1	2%
Static Analysis	WL[21]	1	2%

7. $RQ_{2.4}$ (Modeling & Validation Methods)

The most used architecture modeling language is UML (17%), while alternatives (2% each) remain underexplored. Most studies (74%) lack formal architectural modeling and (87%) do not contain information on the use of MDE. ATAM, SAAM, and static analysis are the only validation methods reported, while 93% of the studies do not report any evaluation strategy, indicating a lack of systematic validation for AI-generated architectural output.

4.3.5. *Generative AI for software architecture: In which cases ($RQ_{2.5}$)*

This subsection presents the specific use cases in which GeneAI has been applied to the software architecture. We examine the types of systems analyzed, the domains in which LLMs are deployed, and the programming languages associated with these use cases. Table 19 presents the use cases and systems addressed in the research papers that apply GenAI to software architecture. According to Table 19, Requirements and Architectural Snippets are the most common subjects, appearing in 15% (7) of research papers, which indicates that LLMs are widely tested in fragments of architectural information WL[6], WL[23]. Enterprise and Property Software and IoT, and Smart Systems also attract significant interest, indicating applications in industrial and network environments. For example, WL[30] used LLMs to reengineer a legacy system at Volvo Group. Since it is challenging to retrieve large-scale open source systems or to evaluate prioritized mobile applications and embedded systems, our findings evidenced that such domains are underrepresented in our study. For example, GL[1] experimented with RAG to evaluate green software patterns starting from architectural documents from Instagram, WhatsApp, Dropbox, Uber, and Netflix. Similarly, WL[26] investigated the architectural reconstruction of an Android app. Finally, 38% (18) of the research articles did not specify a precise use case, that is, position or vision articles.

Table 20 presents the programming languages of the use cases examined. As is evident from Table 20, the most frequent language is Java (7; 13%), reflecting that Java systems are leading the research on LLM applications in software architecture. Other languages, including JavaScript, Python, UML, and Natural Language (NL), occur to a smaller extent, reflecting a mix of implementation and design-level notation.

A significant 58% (30) of the articles did not report the programming language of the use case, and this is an area of reporting that hinders the measurement of LLM uptake by the technology stacks. The presence of legacy languages such as COBOL (1; 2%) suggests that there is research on legacy systems, but only in a very limited subset of cases.

Table 19
Use cases and systems analyzed - ($RQ_{2.5}$).

Category	PaperID	Count[a]	%
Unspecified	WL[7], WL[8], WL[10], WL[13], WL[14], WL[19], WL[22], WL[16], WL[29], WL[32], GL[2], GL[3], GL[4], GL[9], GL[5], GL[6], GL[7], GL[8]	18	38%
Requirement and Architectural Snippets	WL[3], WL[4], WL[5], WL[6], WL[23], WL[28], WL[25]	7	15%
Social Media and Large-Scale systems	GL[1]	1	2%
Architectural documents of Instagram, WhatsApp, Dropbox, Uber, Netflix			
Educational and Research Platforms	WL[11]	1	2%
BigBlueButton, JabRef, TEAMMATES, TeaStore			
Cloud and Open-Source Solutions	WL[31], WL[11], WL[20], GL[10]	4	8%
Google Jump-Start Solution, Hadoop HDFS, MediaStore, Multiple Open-Source Projects			
IoT and Smart systems	WL[24], WL[1], WL[17], WL[35]	4	8%
IoT Reference architectures, Smart City IoT System, Smartwatch App, Remote-Controlled Autonomous Car			
Mobile and Layered Applications	WL[26]	1	2%
Layered App (Android)			
Low-Code and Microservices architectures	WL[12], WL[9], WL[21]	3	6%
Low-Code Development Platforms, Microservices in GitHub, TrainTicket Microservice Benchmark			
Monolithic and Traditional systems	WL[2]	1	2%
Monolithic, Single Component			
Enterprise and Proprietary Software	WL[18], WL[27], WL[36], WL[30]	4	8%
Proprietary Enterprise Scenarios, Ordering System, SuperFrog Scheduler, Volvo SCORE System Requirement Snippets, Snippets of Code, Snippet of Architectural Design Records, Architectural Snippets			
Automotive and Embedded systems	WL[15]	1	2%
PX4 (Drone Software)			
Text-Based and Specialized systems	WL[34], WL[33]	2	4%
Text/Aviation System, Software Engineering Exam Traces			

[a] One paper can have more than one use case.

Table 20
Use case programming language - ($RQ_{2.5}$).

Code	PaperID	Count	%
Unspecified	WL[1], WL[7], WL[8], WL[9], WL[10], WL[11], WL[12], WL[35], WL[13], WL[14], WL[18], WL[17], WL[19], WL[8], WL[22], WL[16], WL[24], WL[29], WL[30], WL[31], WL[32], WL[33], GL[1], GL[2], GL[3], GL[9], GL[5], GL[6], GL[7], GL[10]	30	58%
Java	WL[20], WL[21], WL[25], WL[26], WL[27], WL[28], WL[4]	7	13%
Nature Language	WL[3], WL[5], WL[6], WL[23]	4	8%
JavaScript	WL[27], WL[4]	2	4%
Python	WL[36], WL[4]	2	4%
UML	WL[2], WL[34]	2	4%
C++	WL[15]	1	2%
COBOL	GL[4]	1	2%
Node.js	WL[27]	1	2%
React	WL[27]	1	2%
TypeScript	WL[4]	1	2%

One paper can have more than one Programming Language.

8. $RQ_{2.5}$ (Use Cases)

LLMs are most frequently applied to architectural requirements and snippets (15%), with notable usage in enterprise software and IoT systems (8%), while large-scale, mobile, and embedded systems are less explored. Moreover, Java (13%) is the language most commonly used in LLM-driven architectural studies, but 58% of the studies do not specify a programming language, highlighting a gap in reporting on implementation details.

4.4. Generative AI for software architecture: Challenges (RQ_3)

This subsection presents key challenges identified in the original studies, highlighting limitations in model reliability, ethical concerns, quality of AI-generated outputs, and practical integration issues, which need to be addressed for broader adoption.

Challenges in GenAI research for SA primarily include the **accuracy** of LLM (9; 15%), the most cited issue, emphasizing the difficulty in maintaining accurate and reliable model outputs. LLM **hallucinations** (5; 8%) also represent a critical issue, necessitating mechanisms to prevent incorrect or misleading responses (Table 21 - $\mathbf{RQ_4}$).

Ethics-related concerns (4; 7%), **privacy/data privacy** (7; 12%), and **human interaction** with LLM (3; 5%) indicate that aligning AI outputs with responsible and interpretable practices is increasingly important. Specifically, GL[9] highlights ethical considerations as a major challenge, with issues such as bias in AI-generated architectural decisions and lack of transparency in model reasoning posing significant concerns, particularly in critical domains like healthcare or finance. GL[6] and GL[5] additionally emphasize privacy risks associated with inadvertent information leakage, advocating stronger data protection mechanisms. Addressing these ethical and privacy challenges requires regulatory frameworks, improved model interpretability, and robust security measures.

Moreover, the absence of standardized and task-specific evaluation metrics remains a significant barrier, hindering systematic validation and refinement of AI-generated architectural decisions. Limited context-awareness due to fragmented and inconsistent inputs and inadequate explainability mechanisms further compounds the challenges. Current GenAI models struggle with long-context reasoning and inherent uncertainties such as rigidity in post-hoc adjustments of outputs, persistent hallucinations, and bias, which compromise reliability and trust.

Architectural degradation risks due to overuse or blind trust in AI-generated recommendations necessitate rigorous human oversight and verification processes. Additionally, there is a clear need for architecture-specific datasets, benchmarks, and clearer **semantic traceability** between architectural artifacts to rigorously test and compare model effectiveness.

Table 21
Challenges - (RQ_3).

Code	PaperID	Count[a]	%
LLM Accuracy	WL[4], WL[11], WL[15], WL[18], WL[17], WL[31], GL[6], GL[9], GL[5]	9	15%
Unspecified	WL[30], WL[23], WL[6], WL[25], WL[19], WL[14], GL[3], GL[10], GL[4]	9	15%
LLM Hallucinations	WL[7], WL[27], WL[34], GL[1], GL[9]	5	8%
Ethical Considerations	GL[6], GL[9], GL[5], GL[2]	4	7%
Privacy	WL[34], GL[6], GL[9], GL[5]	4	7%
Architectural Solution Validation	WL[10], WL[26], WL[15], WL[28]	4	7%
Data Privacy	WL[34], GL[6], GL[5]	3	5%
Generated Code Maintainability	WL[21], WL[29]	2	4%
Generated Code Quality	WL[22], WL[5], WL[15]	3	5%
LLM Human Interaction	WL[10], WL[32], WL[2]	3	5%
Traceability	WL[35], WL[36] GL[8]	3	5%
Generated Code Security	WL[4] GL[2]	2	4%
LLM Output Generalizability	WL[16], WL[3]	2	4%
Reduced Human Creativity	GL[2], GL[7]	2	4%
Pattern Recognition Accuracy	GL[1]	1	2%
Intellectual Property	WL[2]	1	2%

[a] One paper can have more than one future challenge.

Quality of generated code, **maintainability** (2; 4%) WL[29], WL[1], **scalability**, and **security** (2; 4%) WL[4], WL[2] represent important areas needing attention. LLM output generalizability (2; 4%) WL[16], WL[3], **reduced human creativity** (2; 4%) GL[7], GL[2], **pattern recognition accuracy** (2; 4%) WL[28], and **intellectual property** concerns (1; 2%) WL[2] also pose significant challenges that must be explicitly considered.

Formal verification and compliance checking have also been suggested as necessary steps to ensure that AI-generated outputs meet defined architectural and regulatory standards GL[7]. In general, studies highlight accuracy, hallucinations, ethics, and practical integration as critical concerns, suggesting a strong need for systematic validation approaches for the generated architectural solutions, enhanced explainability, and rigorous benchmarks for future GenAI adoption in software architecture. Surprisingly, 15% (9) of studies did not mention challenges explicitly, indicating gaps in recognizing and addressing limitations of LLMs in software architecture.

9. RQ_3 (Challenges)

LLM accuracy (15%), hallucinations (9%), and ethical concerns (7%) dominate, alongside critical challenges in evaluation metrics, context-awareness, explainability, systematic validation methods, generalizability, intellectual property, and formal verification.

5. Discussion

This section discusses the implications and elaborates on future directions. Moreover, it identifies the relationships among challenges and future direction and discusses the key differences between white and grey literature findings.

5.1. Implications

The assessed literature has various implications and suggests future direction as follows:

AI-assisted programming: It is an excellent opportunity for short-term future direction. Yet, the products have to be explainable, especially in terms of architecture decisions; this correlates with the need for AI products to generate models or graphs like UML sketches to explain to practitioners the proposed products. There are multiple directions and implications we will elaborate on WL[16].

Integration across SALC phases: Advancement can be claimed once a complete, single integrated GenAI for engineering product development engages in all SALC phases. Currently, we see pieces of the puzzle not necessarily related to the previous phases. An advancement would be to create a framework guiding the integration of all the various tools contributing to one entity or process GL[8].

Evolution, Continuous Architecture, Integration with DevOps: Once we deploy GenAI to manage code, there must be reinforcement learning for architecture optimization, and this must take into account the current trends in software systems, such as cloud-native that employs decentralized architecture (Lelovic et al., 2024). Future perspectives might consider tooling that adjusts the systems to their usage, integrating with DevOps by monitoring user requests and trends by tracing and taking into account available hardware resources or their financial costs. However, GenAI support for system evolution must cope with hallucinations, architecture degradation, and given that we are currently dealing with the pieces of a puzzle with GenAI tools rather than a comprehensive framework for the complete SALC, there is a long path to this.

Legacy Systems: Some major challenges for GenAI are the inability to pivot legacy applications and systems GL[5], GL[6]. Training of GenAI on data and training input is threatened by privacy and intellectual property violations GL[5], GL[6].

Documentation might become legacy: While writing documentation can be expedited by GenAI WL[27], will this still be needed in the future? AI can provide interactive documentation by reverse engineering the code or using other static analysis approaches like those presented by Quevedo et al. WL[21]. Currently, documentation generation requires human intervention to ensure the usefulness, correctness, and validity of the text, and hallucination in evolving systems can be difficult to overcome WL[21].

Cross-team Decentralized Collaboration with AI: Future vision must be elaborated on human-centered cross-team collaboration. For instance, in microservices, we deal with a lot of co-changes that involve various teams (Lelovic et al., 2024). One cannot ignore the fact that most current systems run on a decentralized architecture connecting codebases, where consistency is essential when changes take place to limit ripple effects. Moreover, many issues emerging from the GenAI impact are caused by the neglect of the socio-technical problems and human needs and values WL[27]. Could GenAI facilitate communication across teams when co-changes must take place? Chandraraj GL[2] suggests that GenAI might overlook team dynamics and organizational culture in its architectural suggestions. For example, it might propose a complex solution without considering the team's abilities or the availability of developers with technical skills. It could also suggest a solution that technically works but does not align with the organization's broader objectives.

Who manages the generated content: Model-driven development had one core problem: no one wanted to manage the code that was generated, and when one did, the model generation would not work when the system evolves, as it would override the changes. Similar questions should be asked regarding AI-generated code GL[8],WL[36]. Experimental productivity and quality comparison studies between human-generated and AI-generated code in a realistic environment are needed WL[27]. We need to prevent architectural degradation, and thus, architectural metrics need to be in place.

Complex Architectures: The progress in GenAI should move towards complex architecture. While facing limitations with tokens, advancing RAG, multi-agent frameworks are inevitable to support and complex architectural styles such as microservices WL[4].

Heterogeneous Sources of input: such as software architecture documents, tabular data, and technical diagrams, must be considered since most works focus on a single source of truth, and we might more and more consider aspect oriented programming to come into play with GenAI GL[1], C (2019).

Formal Verification and AI-Driven Compliance Checking: It is easy to start using GenAI tools; however, maximizing the potential can be challenging WL[27]. Moreover, it might be difficult to control the tools, and results need to be checked, as practitioners can easily accept suggestions relying on AI as an oracle. Still, there were observed limits of GenAI to complex tasks with resulting products in less usable WL[27]. This leads to comprehension issues, which we mentioned with challenges to generate UML-like models or diagrams to guide developers on explainability. The correctness, completeness, and effectiveness of the generated code are open to improvement WL[29],GL[5]. Similar to the mentions in white literature, formal verification or result validation is a worthy direction of research.

Replacement of Human Experts: Literature often mentions that the discipline will move towards a field where human experts manage projects, where GenAI agents can prototype or deliver tasks for them to manage WL[27]. This suggests the opportunity for research on AI tools for project management. AI replacing humans can be approached once we overcome trust and establish evaluation metrics. For instance, Prakash GL[8] suggests GenAI helps developers by that 25% to write code efficiently, fix bugs, and improve software quality. GenAI, as a tool for architects, is not a replacement GL[2], GL[3], as it is not yet ready, lacking in complex business contexts, and failing in subjective judgment, business intuition, and personal accountability. Apart from this, Fujitsu GL[4] recognizes the need for interactive capabilities to verify current application specifications and assess the impact of source code changes. However, it is important to be aware of the challenges and ethical considerations associated with GenAI GL[7]. AI algorithms are trained on data, and this data can be biased. This bias can be reflected in the output of AI models. GenAI tools can make mistakes, and they should not be used to replace human judgment. It is also essential to consider the ethical implications of AI-generated architectural patterns and designs before using them.

5.2. Relationships among challenges

The challenges identified in GenAI for Software Architecture are interconnected, influencing each other and impacting the effectiveness and adoption of these technologies. At the core, **LLM accuracy** and **hallucinations** represent foundational issues, significantly affecting the reliability of GenAI outputs. Without addressing these critical concerns, the usefulness of GenAI in SA is limited, as inaccurate or misleading outputs directly undermine trust and adoption.

Ethical considerations, including potential biases and transparency in decision-making processes, closely interact with **privacy** and **data privacy** challenges, highlighting a shared necessity for secure and responsible use of AI models. Ethical issues and privacy concerns demand a holistic approach, incorporating rigorous standards, compliance, and transparency mechanisms.

Similarly, the **validation of architectural solutions** and **traceability** concerns underline the importance of systematic evaluation metrics and clear semantic relationships among architectural artifacts. Effective **validation** and **traceability** practices directly contribute to improved **maintainability**, **security**, and **quality of generated code**. These aspects are integral to maintaining the long-term health and scalability of software systems developed using GenAI.

The **generalizability** of LLM outputs also intersects significantly with accuracy, as enhanced generalizability demands higher context-awareness and improved pattern recognition capabilities. However, excessive reliance on automated solutions introduces the risk of **reduced human creativity**, emphasizing the need for balanced human interaction and oversight.

Intellectual property concerns further underline the necessity of clear guidelines and verification methods to ensure that generated outputs comply with existing legal frameworks, preventing misuse and infringement.

In summary, these challenges collectively highlight the necessity for integrated solutions that comprehensively address accuracy, ethics, privacy, validation, and human interaction, fostering reliable, secure, and ethically responsible application of GenAI technologies in software architecture.

5.3. Relationships between challenges and future directions

The identified challenges from the MLR highlight key concerns influencing the future directions of GenAI within software architecture. Addressing these challenges can guide the practical implications for software architecture practice and research.

The primary challenges emphasized in the literature involve accuracy and reliability, specifically the frequent issue of LLM accuracy and hallucinations. The implications of these challenges point directly to the necessity of enhancing systematic validation approaches for AI-generated architectural solutions, including architectural solution validation. The requirement for precise and robust validation frameworks emerges clearly as LLMs' outputs significantly influence architectural decisions. Formal verification and AI-driven compliance checking have been suggested explicitly to mitigate risks arising from incorrect or misleading AI recommendations.

Ethical considerations and privacy are also prominently discussed challenges, highlighting the importance of integrating ethical AI practices. Concerns about biases, transparency, and inadvertent data leakages underscore the critical need for regulatory frameworks, improved model interpretability, and stronger data protection mechanisms. Ethical and privacy implications strongly suggest the necessity of considering human values and socio-technical dynamics when integrating GenAI tools into architectural practices.

Challenges related to human interaction with LLM, reduced human creativity, and intellectual property concerns further highlight the delicate balance required between automation and human oversight. GenAI must support rather than replace human expertise, as indicated by implications calling for AI-assisted programming to generate explainable artifacts like UML diagrams. This aligns with practical needs, emphasizing that architectural documentation generation currently requires human intervention to ensure accuracy and usability.

The challenge of maintaining the quality of generated code, maintainability, scalability, and security stresses the critical role of human verification processes in the short-term future, particularly when GenAI supports tasks traditionally managed by humans. The implication here is a clear call for experimental productivity and quality studies comparing human-generated and AI-generated code in realistic environments to better understand trade-offs and risks, particularly regarding architectural degradation. Generated code quality and maintainability have implications for comprehensive frameworks integrating DevOps and continuous architecture.

Furthermore, the highlighted challenges related to the generalizability of LLM outputs and traceability suggest the need for enhanced context-awareness and semantic traceability. Addressing these issues aligns with the implications around integrating GenAI across the SALC phases, requiring comprehensive frameworks that manage complex architectures, including continuous architecture and DevOps integration. Specifically, the generalizability of LLM outputs has direct implications for multi-agent frameworks and complex architectures.

Lastly, the challenge of pattern recognition accuracy and managing heterogeneous sources of inputs (e.g., handling multimodal data such as UML diagrams, text in natural language, source code) aligns with implications emphasizing the need for multi-agent frameworks and RAG to handle complex, decentralized architectures such as microservices effectively.

In summary, addressing these highlighted challenges necessitates systematic validation methods, regulatory frameworks, ethical considerations, enhanced human oversight, rigorous comparative studies, and comprehensive integration frameworks. Integrating these insights will be pivotal for effectively realizing the potential of GenAI in software architecture.

5.4. Differences between white and grey literature findings

Our study, being an MLR, covered both the white and grey literature to explore GenAI for Software Architecture. The findings revealed notable differences between these two sources. More specifically, the white literature, including peer-reviewed conference papers and journal papers, mainly addresses formalizing and generalizing the contribution of LLMs to formal software architecture processes. The white literature focused on **LLMs to automate or facilitate architectural decision making**, **traceability**, and model-driven development. Such studies tend to present systematic experiments, propose new methods, or present conceptual foundations to bring LLM into software architecture activities. Moreover, it tends to investigate empirical aspects of LLM use, such as how good they are at generating architectural fragments or determining architectural conformance to predefined standards.

The grey literature comprises blog posts, industry reports, preprints, and white papers and has a more pragmatic and timely focus. LLMs are typically being researched as **work productivity tools** in contrast to scientific objects of intense investigation. Many sources in the grey literature portray LLMs as assistants that assist in making ongoing software development efforts more straightforward, that is, **architecture reconstruction, mapping requirements to architectures, and generating documentation**. The ability of LLMs to act as architectural design copilots, providing quick recommendations or insight versus delving deeper into analytical reasoning, is predominantly what these resources highlight. In contrast to white literature, grey literature features industry-led use cases, for example, using LLMs to plan modernization, automate software lifecycles, and extract knowledge from current codebases.

As expected, the main difference is in the **assessment approach**: the white literature rigorously analyzes the performance of LLM through empirical research, controlled experiments, and case studies, while the grey literature must suffice with anecdotal evidence or high-level summaries without formal endorsement. Moreover, the white literature is more interested in probing theoretical questions, such as the interpretability and trustworthiness of architectural knowledge generated by LLM. In contrast, grey literature tends to be positive and introduces LLMs as enablers without critically addressing their limitations.

In general, both types of literature promote knowledge of LLM implementation in software architecture but differ concerning the purpose and level of critique. The white literature is more research-focused and methodologically clear, and its purpose is to refine and establish LLM integration within the architecture process. The grey literature offers a rapid path to industry learning, whose goal is adoption, tool reviews, and short-term benefit. Since technological hype is a mixture of academic and industry interests, we performed this MLR to capture both worlds and to present a complementary view on the state of the art.

6. Threats to validity

The results of an MLR may be subject to validity threats, mainly concerning the correctness and completeness of the survey. We have structured this Section as proposed by Wohlin (2014), including construct, internal, external, and conclusion validity threats.

Construct validity. Construct validity is related to the generalization of the result to the concept or theory behind the study execution (Wohlin, 2014). In our case, it is related to the potentially subjective analysis of the selected studies. As recommended by Kitchenham's guidelines (Kitchenham and Charters, 2007), data extraction was performed independently by two or more researchers and, in case of discrepancies, a third author was involved in the discussion to clear up any disagreement. Moreover, the quality of each selected paper was checked according to the protocol proposed by Dybå and Dingsøyr (2008).

Internal validity. Internal validity threats are related to possible wrong conclusions about causal relationships between treatment and outcome (Wohlin, 2014). In the case of secondary studies, internal validity represents how well the findings represent the findings reported in the literature. To address these threats, we carefully followed the tactics proposed by Kitchenham and Charters (2007).

External validity. External validity threats are related to the ability to generalize the result (Wohlin, 2014). In secondary studies, external validity depends on the validity of the selected studies. If the selected studies are not externally valid, the synthesis of its content will not be valid either. In our work, we were not able to evaluate the external validity of all the included studies.

Conclusion validity. Conclusion validity is related to the reliability of the conclusions drawn from the results (Wohlin, 2014). In our case, threats are related to the potential non-inclusion of some studies. To mitigate this threat, we carefully applied the search strategy, performing the search in eight digital libraries in conjunction with the snowballing process (Wohlin, 2014), considering all the references presented in the retrieved papers, and evaluating all the papers that reference the retrieved ones, which resulted in one additional relevant paper. We applied a broad search string, which led to a large set of articles, but enabled us to include more possible results. We defined inclusion and exclusion criteria and applied them first to the title and abstract. However, we did not rely exclusively on titles and abstracts to establish whether the work reported evidence of architectural degradation. Before accepting a paper based on title and abstract, we browsed the full text, again applying our inclusion and exclusion criteria.

7. Conclusions

This study presents the results of a multivocal review of the literature investigating the topic of LLM and GenAI applications in the domain of software architecture. It investigated the various perspectives of such practices, including the rationales for applying different LLM models and approaches, application contexts in the software architecture domain, use cases, and potential challenges. From four well-recognized academic literature sources and the three most popular search engines, it extracted 36 white literature and 10 grey literature.

The analyzed results show that LLMs have mainly been applied to support architectural decision-making and reverse engineering, with the GPT model being the most widely adopted. Meanwhile, few-shot prompting is the most commonly adopted technique when human interaction is involved in most studies. Requirement-to-code and Architecture-to-code are the SALC phases where LLMs are mostly applied, while monolith and microservice architectures are the ones that draw the most attention in terms of structured refactoring and anti-pattern detection. Furthermore, the LLM use cases spread from enterprise software and IoT systems to large-scale mobile and embedded systems, where Java is the most commonly used programming language in such studies. However, LLMs also suffer from issues such as accuracy and hallucinations, with other broader issues that need to be addressed in the future.

Our study systematically synthesizes the current practice of LLM adoption in the software architecture domain, which shows clearly that LLM can contribute greatly to helping software architects in various aspects. It is optimistic that LLM, with fast-paced iterative updates, can continue to contribute to this domain with even more astonishing outcomes.

CRediT authorship contribution statement

Matteo Esposito: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation. **Xiaozhou Li:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Data curation. **Sergio Moreschini:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Noman Ahmad:** Investigation, Formal analysis, Data curation. **Tomas Cerny:** Writing – review & editing, Writing – original draft, Funding acquisition, Formal analysis, Conceptualization. **Karthik Vaidhyanathan:** Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Formal analysis. **Valentina Lenarduzzi:** Writing – review & editing, Writing – original draft, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Davide Taibi:** Writing – review & editing, Writing – original draft, Methodology, Funding acquisition, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used ChatGPT to improve language and readability. After using this service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The research presented in this article has been partially funded by the Business Finland Project 6GSoft, by the Academy of Finland project MUFANO/349488 and by the National Science Foundation (NSF), United States Grant No. 2409933.

Data availability statement

We provide our raw data, and the MLR workflow in our replication package hosted on Zenodo https://doi.org/10.5281/zenodo.15032395.

References

Alsayed, A.S., Dam, H.K., Nguyen, C., 2024. Microrec: Leveraging large language models for microservice recommendation. In: Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24, pp. 419–430.

Bucaioni, A., Weyssow, M., He, J., Lyu, Y., Lo, D., 2025. Artificial intelligence for software architecture: Literature review and the road ahead. arXiv:2504.04334.

C, T., 2019. Aspect-oriented challenges in system integration with microservices, soa and iot. Enterp. Inf. Syst. 13, 467–489.

Corbin, J., Strauss, A., 2008. Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory, third ed. SAGE Publications, Inc..

Dybå, T., Dingsøyr, T., 2008. Empirical studies of agile software development: A systematic review. Inf. Softw. Technol. 50, 833–859.

Esposito, M., Palagiano, F., Lenarduzzi, V., Taibi, D., 2024a. Beyond words: On large language models actionability in mission-critical risk analysis. In: International Symposium on Empirical Software Engineering and Measurement. ESEM 2024, pp. 517–527.

Esposito, M., Palagiano, F., Lenarduzzi, V., Taibi, D., 2024b. Beyond words: On large language models actionability in mission-critical risk analysis. In: Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2024, Barcelona, Spain, October 24-25, 2024. ACM, pp. 517–527.

Esposito, M., Palagiano, F., Lenarduzzi, V., Taibi, D., 2024c. On large language models in mission-critical IT governance: Are we ready yet? arXiv preprint arXiv:2412.11698.

Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M., 2023. Large language models for software engineering: Survey and open problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering. ICSE-FoSE, IEEE, pp. 31–53.

Garousi, V., Felderer, M., Mäntylä, M.V., 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. Inf. Softw. Technol. 106, 101–121.

Gustrowsky, B., Villarreal, J.L., Alférez, G.H., 2024. Using generative artificial intelligence for suggesting software architecture patterns from requirements. In: Arai, K. (Ed.), Intelligent Systems and Applications. Springer Nature Switzerland, Cham, pp. 274–283.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2024. Large language models for software engineering: A systematic literature review. ACM Trans. Softw. Eng. Methodol. 33, 1–79.

Jiang, J., Wang, F., Shen, J., Kim, S., Kim, S., 2024. A survey on large language models for code generation. arXiv preprint arXiv:2406.00515.

Kaplan, A., Keim, J., Schneider, M., Koziolek, A., Reussner, R., 2024. Combining knowledge graphs and large language models to ease knowledge access in software architecture research.

Kitchenham, B., Brereton, P., 2013. A systematic review of systematic review process research in software engineering. Inf. Softw. Technol. 55, 2049–2075.

Kitchenham, B., Charters, S., 2007. Guidelines for performing systematic literature reviews in software engineering.

Lelovic, L., Huzinga, A., Goulis, G., Kaur, A., Boone, R., Muzrapov, U., Abdelfattah, A.S., Cerny, T., 2024. Change impact analysis in microservice systems: A systematic literature review. J. Syst. Softw. 112241.

Marques, N., Silva, R.R., Bernardino, J., 2024. Using chatgpt in software requirements engineering: A comprehensive review. Futur. Internet 16, 180.

Ozkaya, I., 2023. Application of large language models to software engineering tasks: Opportunities, risks, and implications. IEEE Softw. 40, 4–8.

Russo, D., 2024. Navigating the complexity of generative ai adoption in software engineering. ACM Trans. Softw. Eng. Methodol. 33, 1–50.

Santos, P.d.O., Figueiredo, A.C., Nuno Moura, P., Diirr, B., Alvim, A.C., Santos, R.P.D., 2024. Impacts of the usage of generative artificial intelligence on software development process. In: Proceedings of the 20th Brazilian Symposium on Information Systems. pp. 1–9.

Saucedo, A., Rodríguez, G., 2024. Migration of monolithic systems to microservices using ai: A systematic mapping study. In: Anais do XXVII Congresso Ibero-Americano em Engenharia de Software. SBC, pp. 1–15.

Sauvola, J., Tarkoma, S., Klemettinen, M., Riekki, J., Doermann, D., 2024. Future of software development with generative ai. Autom. Softw. Eng. 31, 26.

Schmid, L., Hey, T., Armbruster, M., Corallo, S., Fuchß, D., Keim, J., Liu, H., Koziolek, A., 2025. Software architecture meets llms: A systematic literature review.

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q., 2024. Software testing with large language models: Survey, landscape, and vision. IEEE Trans. Softw. Eng. 50, 911–936.

Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: EASE 2014.

Selected White Literature

[WL1] Adnan, B., Miryala, S., Sambu, A., Vaidhyanathan, K., De Sanctis, M., Spalazzese, R., 2025. Leveraging llms for dynamic iot systems generation through mixed-initiative interaction. In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, IEEE Computer Society, Los Alamitos, CA, USA, pp. 488–497.

[WL2] Ahmad, A., Waseem, M., Liang, P., Fahmideh, M., Aktar, M.S., Mikkonen, T., 2023. Towards human-bot collaborative software architecting with chatgpt. In: Proceedings of the International Conference on Evaluation and Assessment in Software Engineering. EASE'23, ACM, New York, NY, USA, p. 7.

[WL3] Arias, S., Suquisupa, A., Granda, M.F., Saquicela, V., 2024. Generation of Microservice Names from Functional Requirements: An Automated Approach. Springer Nature Switzerland, Cham, pp. 157–173.

[WL4] Arun, S., Tedla, M., Vaidhyanathan, K., 2025. LLMs for generation of architectural components: An exploratory empirical study in the serverless world. In: 2025 IEEE 22nd International Conference on Software Architecture. ICSA, IEEE Computer Society, Los Alamitos, CA, USA, pp. 25–36.

[WL5] Dhar, R., Vaidhyanathan, K., Varma, V., 2024a. Can llms generate architectural design decisions? - an exploratory empirical study. In: 2024 IEEE 21st International Conference on Software Architecture. ICSA, pp. 79–89.

[WL6] Dhar, R., Vaidhyanathan, K., Varma, V., 2024b. Leveraging generative ai for architecture knowledge management. In: 2024 IEEE 21st International Conference on Software Architecture Companion. ICSA-C, pp. 163–166.

[WL7] Díaz-Pace, J.A., Tommasel, A., Capilla, R., 2024. Helping novice architects to make quality design decisions using an llm-based assistant. In: European Conference on Software Architecture. Springer, pp. 324–332.

[WL8] Diaz-Pace, J.A., Tommasel, A., Capilla, R., Ramirez, Y.E., 2025. Architecture exploration and reflection meet llm-based agents. In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, IEEE Computer Society, Los Alamitos, CA, USA, pp. 1–5.

[WL9] Duarte, C.E., 2025. Automated microservice pattern instance detection using infrastructure-as-code artifacts and large language models. In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, IEEE Computer Society, Los Alamitos, CA, USA, pp. 161–166.

[WL10] Eisenreich, T., Speth, S., Wagner, S., 2024. From requirements to architecture: An ai-based journey to semi-automatically generate software architectures. In: Proceedings of the 1st International Workshop on Designing Software. pp. 52–55.

[WL11] Fuchs, D., Liu, H., Hey, T., Keim, J., Koziolek, A., 2025. Enabling architecture traceability by llm-based architecture component name extraction. In: 2025 IEEE 22nd International Conference on Software Architecture. ICSA, IEEE Computer Society, Los Alamitos, CA, USA, pp. 1–12.

[WL12] Hagel, N., Hili, N., Bartel, A., Koziolek, A., 2025. Towards llm-powered consistency in model-based low-code platforms. In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, IEEE Computer Society, Los Alamitos, CA, USA, pp. 364–369.

[WL13] Ivers, J., Ozkaya, I., 2025. Will generative ai fill the automation gap in software architecting? In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, IEEE Computer Society, Los Alamitos, CA, USA, pp. 41–45.

[WL14] Jahić, J., Sami, A., 2024. State of practice: Llms in software engineering and software architecture. In: 2024 IEEE 21st International Conference on Software Architecture Companion. ICSA-C, pp. 311–318.

[WL15] Johansson, N., Caporuscio, M., Olsson, T., 2024. Mapping source code to software architecture by leveraging large language models. In: Software Architecture. ECSA 2024 Tracks and Workshops. Springer Nature Switzerland, Cham, pp. 133–149.

[WL16] Larsen, K.R., Edvall, M., 2024. Investigating the impact of generative ai on newcomers' understanding of software projects.

[WL17] Lutze, R., Waldhör, K., 2024. Generating specifications from requirements documents for smart devices using large language models (llms). In: Kurosu, M., Hashizume, A. (Eds.), Human-Computer Interaction. Springer Nature Switzerland, Cham, pp. 94–108.

[WL18] Maranhã, J.a.J., Guerra, E.M., 2024. A prompt pattern sequence approach to apply generative ai in assisting software architecture decision-making. In: Proceedings of the 29th European Conference on Pattern Languages of Programs, People, and Practices. EuroPLoP '24, Association for Computing Machinery, New York, NY, USA.

[WL19] Miñ, J., Andrade, R., Torres, J., Chicaiza, K., 2024. Leveraging generative artificial intelligence for software antipattern detection. In: Li, S. (Ed.), Information Management.

[WL20] Pandini, G., Martini, A., Videsjorden, A.N., Fontana, F.A., 2025. An exploratory study on architectural smell refactoring using large languages models. In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, pp. 462–471.

[WL21] Quevedo, E., Abdelfattah, A.S., Rodriguez, A., Yero, J., Cerny, T., 2024. Evaluating chatgpt's proficiency in understanding and answering microservice architecture queries using source code insights. SN Comput. Sci. 5, 422.

[WL22] Raghavan, P., 2024. Ipek ozkaya on generative ai for software architecture. IEEE Softw. 41, 141–144.

[WL23] Rejithkumar, G., Anish, P.R., Shukla, J., Ghaisas, S., 2024. Probing with precision: Probing question generation for architectural information elicitation. In: 2024 IEEE/ACM Workshop on Multi-Disciplinary, Open, and RElevant Requirements Engineering. MO2RE, pp. 8–14.

[WL24] Rubei, R., Di Salle, A., Bucaioni, A., 2025. Llm-based recommender systems for violation resolutions in continuous architectural conformance. In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, IEEE Computer Society, Los Alamitos, CA, USA, pp. 404–409.

[WL25] Rukmono, S.A., Ochoa, L., Chaudron, M.R., 2023. Achieving high-level software component summarization via hierarchical chain-of-thought prompting and static code analysis. In: 2023 IEEE International Conference on Data and Software Engineering. ICoDSE, pp. 7–12.

[WL26] Rukmono, S.A., Ochoa, L., Chaudron, M., 2024. Deductive software architecture recovery via chain-of-thought prompting. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results. ICSE-NIER'24, Association for Computing Machinery, New York, NY, USA, pp. 92–96.

[WL27] Saarinen, L., 2024. Generative ai in software development. Inf. Technol..

[WL28] Schindler, C., Rausch, A., 2024. Formal software architecture rule learning: A comparative investigation between large language models and inductive techniques. Electronics 13.

[WL29] Sharma, T., 2024. Llms for code: The potential, prospects, and problems. In: 2024 IEEE 21st International Conference on Software Architecture Companion. ICSA-C, pp. 373–374.

[WL30] Singh, V., Korlu, C., Orcun, O., Assunçao, W.K., 2024. Experiences on using large language models to re-engineer a legacy system at volvo group. Methods 13, 14.

[WL31] Soliman, M., Keim, J., 2025. Do large language models contain software architectural knowledge? : An exploratory case study with gpt. In: 2025 IEEE 22nd International Conference on Software Architecture. ICSA, IEEE Computer Society, Los Alamitos, CA, USA, pp. 13–24.

[WL32] Supekar, V., MIT WPU, P., Khande, R., 2024. Improving software engineering practices: Ai-driven adoption of design patterns.

[WL33] Tagliaferro, A., Corboe, S., Guindani, B., 2025. Leveraging llms to automate software architecture design from informal specifications. In: 2025 IEEE 22nd International Conference on Software Architecture Companion. ICSA-C, IEEE Computer Society, Los Alamitos, CA, USA, pp. 291–299.

[WL34] Tang, S., Chen, X., Xiao, H., Wei, J., Li, Z., 2023. Using problem frames approach for key information extraction from natural language requirements. In: 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion. QRS-C, pp. 330–339.

[WL35] Von Heissen, O., Hanke, F., Mpidi Bita, I., Hovemann, A., Dumitrescu, R., et al., 2024. Toward intelligent generation of system architectures. In: DS 130: Proceedings of NordDesign 2024, Reykjavik, Iceland, 12th-14th August 2024. pp. 504–513.

[WL36] Wei, B., 2024. Requirements are all you need: From requirements to code with llms. In: 2024 IEEE 32nd International Requirements Engineering Conference. RE, IEEE, pp. 416–422.

Selected Gray Literature

[GL1] Ahuja, N., Feng, Y., Li, L., Malik, A., Sivayoganathan, T., Balani, N., Rakhunathan, S., Sarro, F., 2024. Automatically assessing software architecture compliance with green software patterns. URL: https://solar.cs.ucl.ac.uk/pdf/EcoDocSense_Greens2025.pdf.

[GL2] Chandraraj, K., 2023. Generative ai in software architecture: Don't replace your architects yet. Medium URL: https://medium.com/inspiredbrilliance/generative-ai-in-software-architecture-dont-replace-your-architects-yet-cde0c5d462c5, (Accessed 02 March 2025).

[GL3] Data Within Reach, 2023. The future of software architecture: Diagrams as code (dac). YouTube, URL: https://www.youtube.com/watch?v=4Q5koGd1XGA, (Accessed 02 March 2025).

[GL4] Fujitsu, 2025. Fujitsu Launches Gen Ai Software Analysis and Visualization Service to Support Optimal Modernization Planning. Press Release, URL: https://www.fujitsu.com/global/about/resources/news/press-releases/2025/0204-01.html, (Accessed 02 March 2025).

[GL5] Martelli, K., Cao, H., Cheng, B., 2023. Generative Ai and the Software Development Lifecycle. KPMG Report, SDLC, URL: https://kpmg.com/kpmg-us/content/dam/kpmg/pdf/2023/KPMG-GenAI-and-SDLC.pdf, (Accessed 02 March 2025).

[GL6] Nandi, A., 2024. Gen ai in software development: Revolutionizing the planning and design phase. AIM Res. URL: https://aimresearch.co/council-posts/gen-ai-in-software-development-revolutionizing-the-planning-and-design-phase, (Accessed 02 March 2025).

[GL7] Paradkar, S., 2023. Software architecture and design in the age of generative ai: Opportunities, challenges, and the road ahead. Medium URL: https://medium.com/oolooroo/software-architecture-in-the-age-of-generative-ai-opportunities-challenges-and-the-road-ahead-d410c41fdeb8, (Accessed 02 March 2025).

[GL8] Prakash, M., 2024. Role of Generative AI Tools (GAITs) in Software Development Life Cycle (SDLC)-Waterfall Model. Massachusetts Institute of Technology.

[GL9] Rivera Hernández, B.M., Santos Ayala, J.M., Méndez Melo, J.A., 2024. Generative ai for software architecture.

[GL10] Seroter, R., 2023. Would Generative Ai Have Made me a Better Software Architect? Probably. Richard Seroter's Blog, URL: https://seroter.com/2023/10/16/would-generative-ai-have-made-me-a-better-software-architect-probably/, (Accessed 02 March 2025).